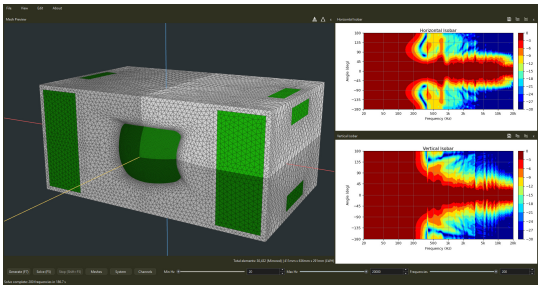
Boundary Lab Guide

This guide was generated automatically from markdown files inside `/docs/`.

Main Window

The main window is a dockable workspace containing:

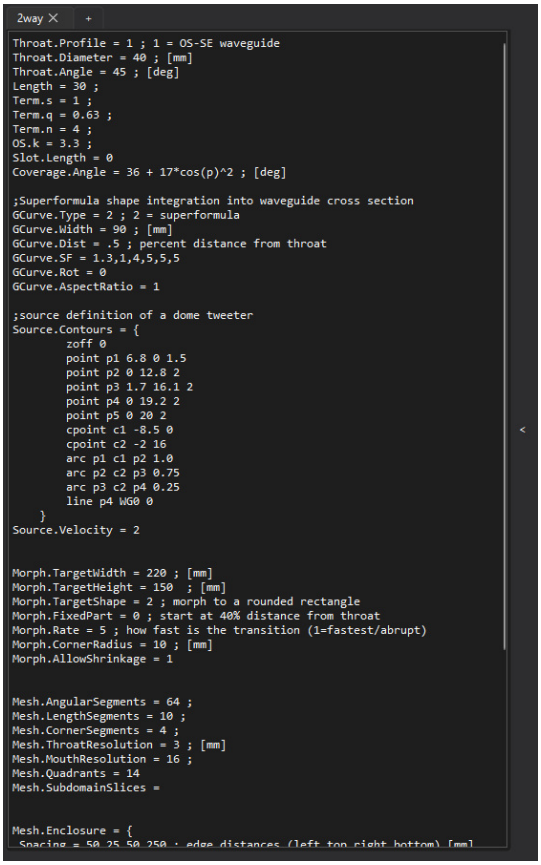
- the waveguide design editor;
- the 3D mesh preview;
- horizontal and vertical isobars, acoustic impedance, on-axis response, and spinorama plots;
- the command strip and status readout.



Panels can be resized, rearranged, floated, or closed. Reopen a closed panel from the **View** menu.

Waveguide Design Editor

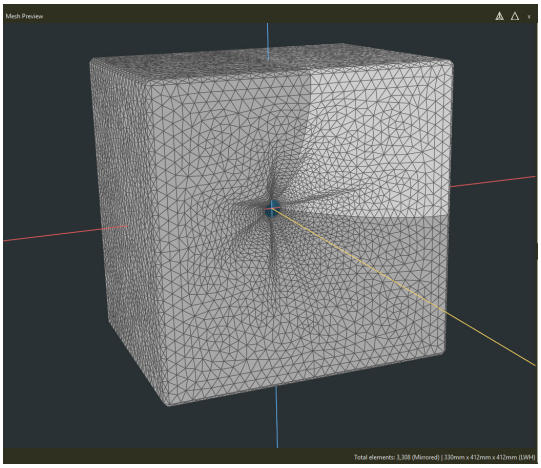
The waveguide design panel contains the editor supplied by the active geometry provider. The bundled Ath provider edits Ath `.cfg` text. Use **File > Import Waveguide Design...** and **Export Waveguide Design...** to exchange the active design with other tools.



Use the tab controls to add, remove, and rename designs. Double-click a tab to rename it. Each enabled design contributes its generated mesh to the project, which makes it possible to assemble multiway models or compare alternatives.

3D Mesh Preview

The preview displays every enabled generated and imported mesh. Rigid surfaces are gray, moving surfaces are blue, and FEM-BEM interface surfaces are green. Mirrored images are shaded darker when X or XY symmetry is active. Hover over a surface to see its mesh name, physical-group name and tag, and element count. The lower-right readout reports the active element count and model dimensions.



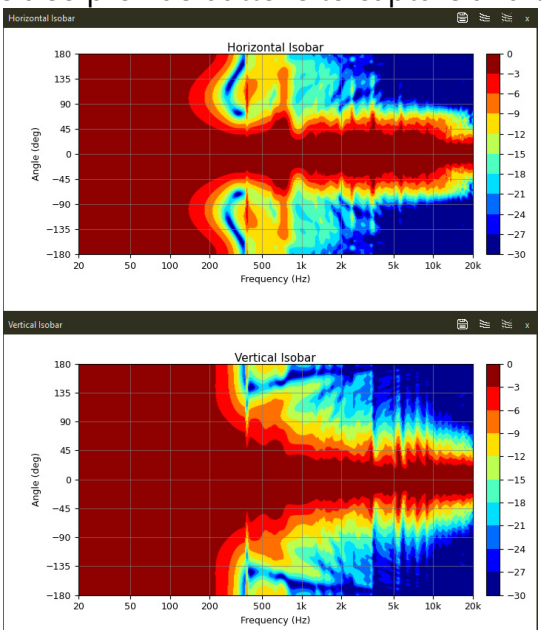
The red, blue, and yellow axes represent X, Y, and Z. The acoustic reference axis is +Z, and

observation points are positioned relative to the global origin. Use **Meshes** to change a mesh's scale or XYZ translation.

For a coupled system, the two buttons in the preview title bar can isolate the bounded interior or unbounded exterior region. They are disabled when the project does not contain the corresponding region type.

Plot Views

Plots are enabled from the **View** menu and update as frequencies complete when live plot streaming is enabled. Use the save button in a plot's title bar to export that plot as a PNG. Horizontal and vertical isobars also provide buttons to capture and clear contour overlays.



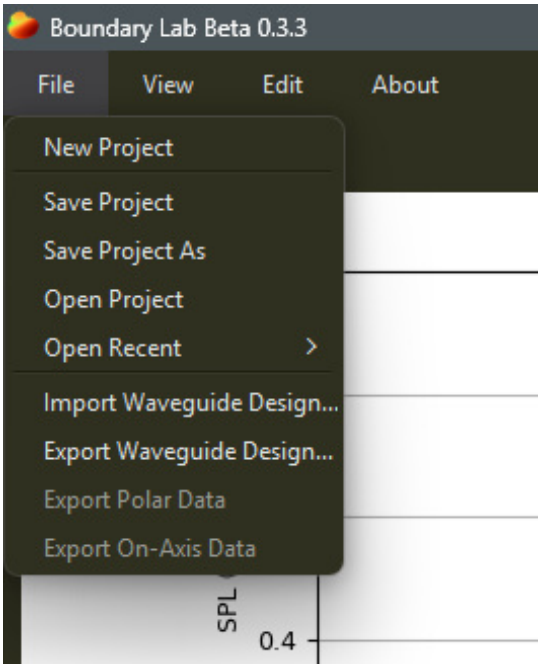
All five plot types share the same mouse interactions:

- Press and drag the left mouse button to position a crosshair. The crosshair remains after release; double-click the plot to remove it.
- Hold the right mouse button to show the previous completed solve. Releasing the button or moving the pointer outside the plot restores the current solve.

The comparison gesture becomes available after a new solve has replaced a previous completed result. Multi-curve plots report the raw cursor coordinates rather than snapping to one curve.

Menu Bar

File Menu

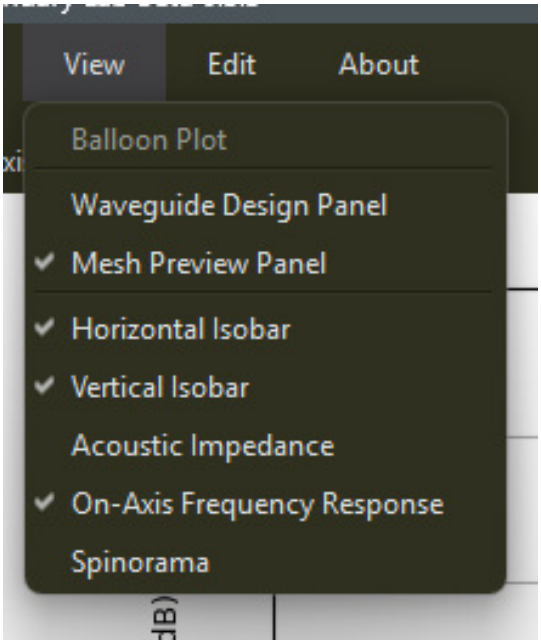


- **New Project** clears the current design, mesh, physical-system, channel, and solved-result state after prompting for unsaved changes.
- **Save Project** writes to the active `.blab.json` path.
- **Save Project As** selects a new project path.
- **Open Project** loads a `.blab.json` project.
- **Open Recent** lists recently opened projects and can clear that history.
- **Import/Export Waveguide Design** reads or writes the active provider's editable design source.
- **Export Polar Data** writes horizontal and vertical response text files.
- **Export On-Axis Data** writes SPL and phase for each solved channel.

All application file and directory pickers share one last-used directory. It is remembered between application sessions, falls back to an existing folder if the saved location disappears, and changes only after a selection is accepted.

Project files do not contain solved results or solver-backend choices. They do contain reproducibility-related observation and visualization preferences; on open, Boundary Lab asks before applying values that differ from the current application settings.

View Menu

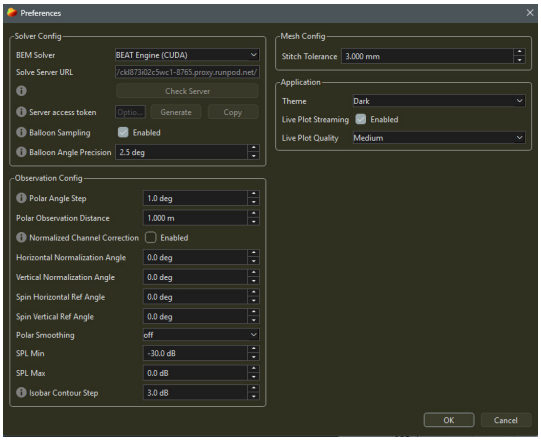


The View menu shows or hides the design editor, mesh preview, and five plot panels. **Balloon Plot** opens its own window and is enabled only when the current solve contains spherical samples.

Edit and About Menus

Edit > Preferences opens application and solve preferences. **About** contains diagnostic information, donation information, and the bundled help guide.

Preferences



Solver Config

- **BEM Solver** selects Server, BEAT Engine CUDA, BEAT Engine CPU, BEAT Engine ROCm, or Bempp OpenCL CPU. ROCm is currently a placeholder. Coupled systems require the local BEAT Engine CPU or CUDA backend.
- **Solve Server URL** and **Check Server** configure and query a remote exterior-BEM server. A successful health check also updates advertised capabilities such as symmetry support.
- **Server access token** is an optional bearer token for an authenticated server. Generate or

paste it, copy it into the deployment's secret store, and keep it safe. Boundary Lab retains it only for the current application session.

- **Balloon Sampling** requests Fibonacci-sphere observation points during the solve. Without these samples, the Balloon Plot action remains unavailable.
- **Balloon Angle Precision** controls the approximate angular spacing and, consequently, the number of solved balloon vertices. The default 2.5-degree spacing produces approximately 6,600 points.

Observation Config

- **Polar Angle Step** controls the solved horizontal and vertical observation spacing. Spinorama processing requires 10-degree spacing or finer.
- **Polar Observation Distance** sets the observation radius from the origin.
- **Normalized Channel Correction** applies a per-channel reference-axis magnitude correction before channel gain, delay, polarity, and crossover filtering.
- **Horizontal/Vertical Normalization Angle** chooses the reference angle used for directivity normalization in each plane.
- **Spin Horizontal/Vertical Ref Angle** chooses the reference axes for the spinorama on-axis and listening-window curves without changing the early reflections or sound-power data.
- **Polar Smoothing** applies fractional-octave smoothing to directivity, spinorama, and balloon presentation data.
- **SPL Min/Max** set the displayed and exported directivity clipping range.
- **Isobar Contour Step** selects stepped isobar colors. Set it to 0 dB for a smoothly interpolated color map.

Mesh Config

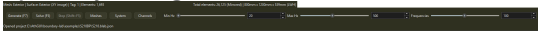
Stitch Tolerance is the maximum distance used when joining adjacent mesh parts for an exterior region whose stitching option is enabled in **System > Regions**.

Application

- **Theme** selects the system, light, or dark appearance.
- **Live Plot Streaming** enables plot refreshes while a solve is running.
- **Live Plot Quality** selects the interpolation density used for those live updates. Completed solves are rendered at final quality.

Command Strip

The command strip contains geometry generation and solve controls, entry points for project configuration, logarithmic frequency-range controls, and the current status message.



Generate

Generate (F7) runs the active design through its geometry provider. With Ath, Boundary Lab stages the `.cfg`, runs `ath.exe` (through Wine when needed), cleans the generated surface mesh, and loads it into the project and preview. Managed generated artifacts are stored below `runs/generated_geometry`.

Solve and Stop

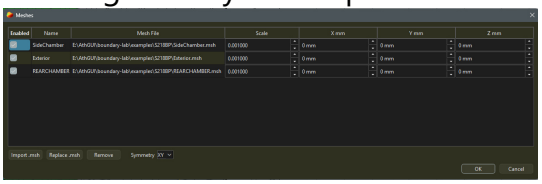
Solve (F5) infers the numerical path from **System > Regions**:

- one unbounded exterior and no bounded regions uses exterior BEM;
- one or more bounded FEM regions plus one unbounded exterior uses the coupled FEM-BEM path.

Stop (Shift+F5) requests cooperative cancellation. A backend may finish the in-flight frequency before stopping; completed frequencies remain available for plotting and export. BEAT Engine CPU and CUDA keep a persistent Julia worker between solves, including after ordinary cancellation, so later solves can reuse the initialized process.

Meshes

The **Meshes** window lists generated geometry and imported `.msh` files.



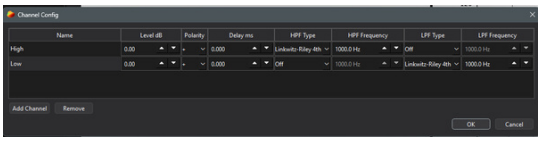
Generated rows are locked to their design documents. Imported rows can be enabled, disabled, renamed, removed, scaled, translated, or replaced. **Replace .msh** is available for one selected imported row and preserves that row's identity so region, boundary, interface, and component references continue to work when the replacement retains the same physical-group names. `.msh` files may also be dragged into the table.

Choose **Off**, **X**, or **XY** symmetry here. Reduced meshes must lie in the positive-X, or positive-X/positive-Y, fundamental domain. Symmetry is available for local BEAT Engine CPU/CUDA and for servers that advertise support.

When the application regains focus, it checks enabled imported source files for external changes. Changed BEM and FEM meshes are reloaded automatically. If a configured interface depends on a changed mesh, Boundary Lab verifies the pair and rebuilds the derived conforming BEM interface mesh when necessary.

Channels

Channels apply post-solve synthesis to independently solved component bases. They are useful for multiway interference and crossover studies.



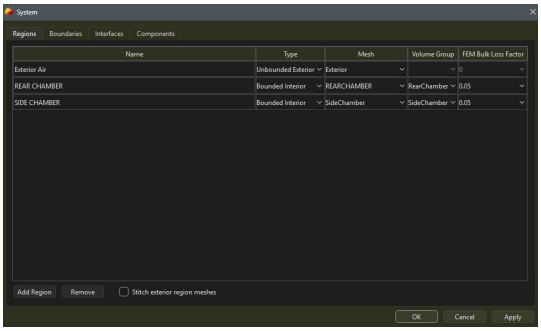
- **Name** identifies the channel used by components.
- **Level dB**, **Polarity**, and **Delay ms** apply complex channel weights.
- **HPF/LPF Type** and **Frequency** define idealized analog crossover transfer functions.

Applying channel changes resynthesizes existing basis results without running the acoustic solver again. The ordinary plots refresh immediately; balloon data is resynthesized only while its window is open.

System

The **System** window is the physical-model editor for both exterior BEM and coupled FEM-BEM projects. Every active surface defaults to **Rigid** and can be changed to **Moving** or **Interface**. The UI has no unassigned or unused surface state.

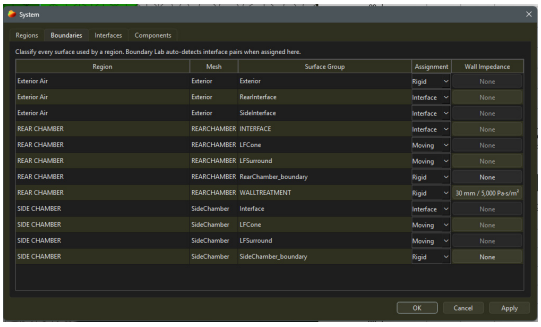
Regions



Create exactly one **Unbounded Exterior** region and assign its BEM surface mesh or meshes. Add a **Bounded Interior** for each FEM chamber, choose its tetrahedral mesh and physical volume group, and optionally select a homogeneous FEM bulk-loss factor. If the exterior uses adjoining parts of one continuous surface, enable **Stitch exterior region meshes**; leave it off for disconnected closed bodies.

An exterior-only system supports prescribed-velocity components. Adding a bounded region selects the coupled path and enables the Interfaces tab.

Boundaries

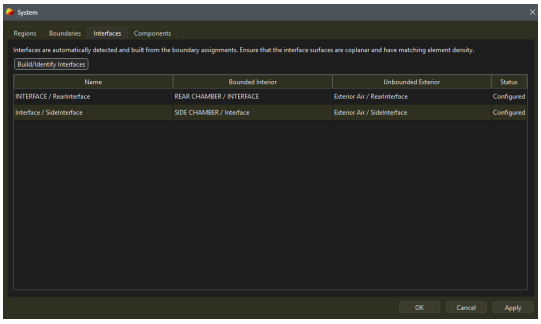


Classify each region surface as **Rigid**, **Moving**, or **Interface**. A bounded rigid surface may

additionally use a rigid-backed porous lining via **Wall Impedance**. The Miki model accepts lining thickness and airflow resistivity; disabling it restores the hard-wall condition.

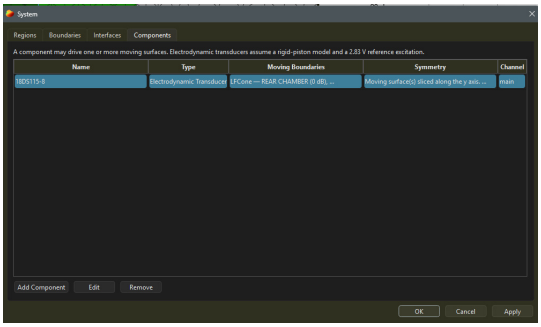
Moving boundaries must be owned by exactly one component. An FEM and BEM port mouth uses two interface boundary assignments, one in each acoustic region.

Interfaces



Build/Identify Interfaces pairs configured bounded and unbounded interface surfaces, makes the BEM side conform to the authoritative FEM boundary facets when needed, and validates node, face, and normal-orientation mappings. The original imported files are not overwritten. Multiple tagged openings may share the same surrounding BEM surface. This tab is disabled when no bounded FEM region exists.

Components



Components own one or more moving boundaries and route their solved reference response to an application channel. The editor supports **Prescribed Velocity** and **Electrodynamic Transducer** components.

Each selected surface has a **Relative Velocity** in dB, allowing a dome and surround to share a component while using different motion amplitudes. Ath-generated driver groups are initially seeded as prescribed-velocity components.

Electrodynamic components use direct Re, Le, Bl, Mmd, Cms, and Rms parameters with a 2.83 V reference excitation. Their rigid-translation motion axis can be inferred from the selected surface normals or entered manually. In a symmetry model, Boundary Lab also infers whether moving surfaces are cut by the active planes and reports how many distinct components exist in the fully mirrored system; there is no manual component-symmetry multiplier.

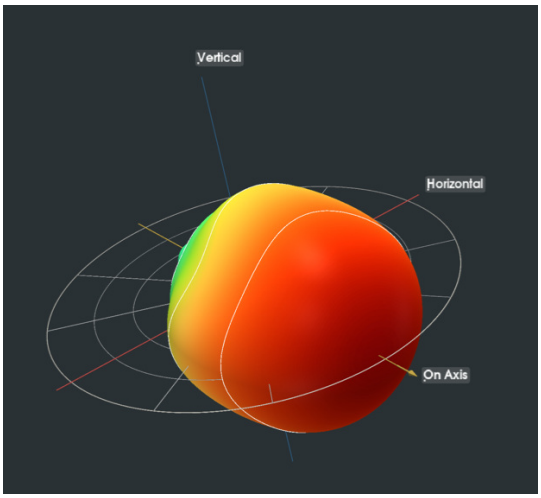
See Physical System Model for the object model and Coupled Solver for numerical requirements and limitations.

Frequency Controls

Set **Min Hz**, **Max Hz**, and **Frequencies** before solving. Boundary Lab uses logarithmic spacing between the normalized minimum and maximum values.

Balloon Plot

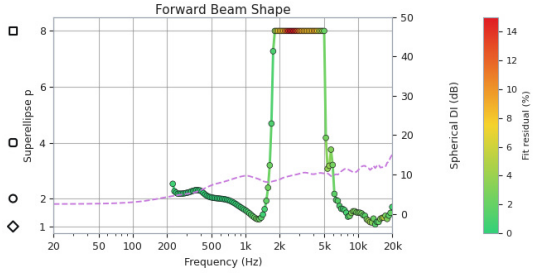
After a solve with spherical sampling enabled, choose **View > Balloon Plot**.



The viewer includes:

- a rotatable and zoomable 3D directivity balloon whose vertices are the original Fibonacci-sphere solve samples;
- a frequency slider, SPL color scale, and 6 dB surface contours that update while the slider is dragged;
- horizontal, vertical, and on-axis guides;
- a rotatable polar protractor with 30-degree spokes and 6 dB rings;
- radar and isobar slice plots for the current frequency and slice angle;
- the Forward Beam Shape diagnostic plot.

The viewer includes every frequency completed before the solve ended. Use its **File > Export Balloon Data** action to write `metadata.json`, `topology.npz`, `spl_db.npy`, and `radius_norm.npy`. The schema preserves the original sample directions, shared triangle topology, frequency axis, normalized SPL, and normalized radius values; XYZ surface positions can be reconstructed from the documented direction and radius mapping. Guide geometry, contours, and slice plots are not included.



See Forward Beam Shape Plot for the meaning and limitations of that diagnostic.

Project Files

Boundary Lab project files use readable JSON and the `.blab.json` extension.

Project files store:

- waveguide design documents, including generator provider ID and provider-owned source
- generated mesh enabled state, scale, and XYZ offset
- imported mesh rows, including absolute `.msh` paths
- whether the exterior region's mesh parts should be stitched into one solve mesh
- an editable physical-system graph for exterior BEM and coupled BEM/FEM models
- application-side component-to-channel routing for excitation responses

Project files do not store:

- solved BEM results
- exported plots
- solver backend, GMRES tolerance, and Burton-Miller preferences
- generated geometry file contents

Example Shape

```
{
  "schema_version": 7,
  "active_generator_document_id": "design1",
  "generator_documents": [
    {
      "id": "design1",
      "name": "Waveguide",
      "provider_id": "ath",
      "provider_schema_version": 1,
      "source": {"format": "ath_cfg", "text": "..."},
      "mesh_enabled": true,
      "mesh_scale_factor": 0.001,
      "mesh_translation_mm": [0, 0, 0],
      "artifact": null
    }
  ],
  "imported_meshes": [],
  "stitch_exterior_meshes": false,
  "physical_system": {},
  "component_channel_by_id": {}
}
```

Loading Projects

Loading a project updates the design editor, mesh, channel, and physical-system configuration. It does not automatically run a geometry provider or start a solve. Older source-config projects are converted to an exterior physical system as soon as their mesh artifacts are available. Pending compatibility assignments are retained when a generated artifact must be rebuilt first. Schema-6 projects that stored a global FEM bulk-loss preference are migrated by copying that value to

every bounded acoustic region.

If the project references imported mesh files, those paths are expected to exist on the local machine. Relative mesh and generated-output paths are resolved from the project file's directory, which keeps bundled samples portable.

The application uses one shared last-used directory for open, save, import, export, and directory-selection dialogs. The directory is stored in QSettings between sessions, is updated only after the user accepts a selection, and falls back to an existing home or working directory if the remembered path is no longer available.

Enabled imported `.msh` sources are checked when the application regains focus. If an external tool has changed a BEM or FEM file, Boundary Lab reloads it. Configured interfaces that depend on the changed mesh are then verified and, when necessary, rebuilt from the refreshed FEM boundary facets. The **Replace .msh** action provides an explicit alternative for imported rows and preserves downstream references when physical-group names remain compatible.

Conforming FEM and BEM Interfaces

Coupled FEM–BEM models require both meshes to use the same nodes and triangle connectivity on their shared interface. Boundary Lab includes a command-line utility for the common case where a tagged planar BEM patch is surrounded by one planar surface:

```
blab conform-interface femvolume.msh exterior.msh exterior_conforming.msh
```

The FEM tetrahedron boundary facets are authoritative. The utility copies those facets into the output BEM mesh, remeshes the surrounding BEM surface to meet their perimeter, preserves the BEM physical groups, and verifies that the resulting exterior mesh is watertight. Input physical-group names default to `Interface` and can be changed with `--fem-interface` and `--bem-interface`. The original input files are not modified.

See Physical System Model for how mesh groups, regions, boundaries, interfaces, components, and excitation ports relate to one another. See Coupled Solver for the initial direct FEM–BEM backend and its supported outputs.

Exporting Plot Images

Use the save button in any plot panel's title bar to export its current figure as a `.png` image. Isobar exports are prepared at final interpolation quality.

Exporting Polar Data

Users can export simulated polar data as individual .txt files per angle sampled for horizontal and vertical axes. Channel-basis solves export three tab-separated columns: frequency in Hz, normalized SPL in dB, and relative phase in degrees. The .txt files can be directly imported into tools such as REW and VituixCAD for external analysis.

Exporting On-Axis Data

Users can export each solved channel's on-axis response as a tab-separated .txt file containing frequency in Hz, SPL in dB, and phase in degrees. Single-channel solves use a save-file dialog. Multi-channel solves use a directory picker and write one file per channel; the combined system response is not exported. The files use the original solved frequency samples and can be imported into tools such as REW and VituixCAD.

Exporting Balloon Data

The balloon viewer exports a schema-versioned directory containing:

- `metadata.json`, including coordinate conventions, array shapes, and radius reconstruction rules;
- `topology.npz`, containing frequency, angular and Cartesian sample directions, and the shared triangular topology;
- `spl_db.npy`, with shape `(frequency, point)`;
- `radius_norm.npy`, with the same shape and values clipped to the configured balloon dB range.

The point order is the original solver Fibonacci-sample order. The export does not contain contour actors, axes, protractor geometry, or radar/isobar slice plots.

Model Assumptions

Boundary Lab is a frequency-domain linear-acoustics application for predicting loudspeaker radiation. It supports two physical-system topologies:

- an unbounded exterior region only, solved with BEM;
- one or more bounded tetrahedral air regions coupled to an unbounded exterior BEM region, with optional lumped electrodynamic transducer equations.

The application infers the path from the regions configured in the **System** window. It is intended to answer questions about acoustic response, directivity, source interaction, enclosure and port behavior, and linear transducer loading. It is not a general structural, thermal, or nonlinear finite-element package.

Physical System and Boundary Conditions

Every active mesh surface belongs to an acoustic region and is classified as:

1. **Rigid**: zero normal surface velocity. A bounded rigid wall may optionally use the supported locally reacting Miki porous-lining treatment.
2. **Moving**: motion is supplied by one prescribed-velocity or electrodynamic component.
3. **Interface**: pressure and normal flux are transferred between a bounded FEM region and the exterior BEM region.

The UI defaults every surface to Rigid and does not provide an unassigned or unused state. This

prevents an omitted assignment from silently behaving as an opening.

An exterior-only model accepts prescribed-velocity components. A coupled model may use prescribed-velocity components, linear electrodynamic transducers, or both. A component can own several moving surface groups, and each group can have a relative velocity weight. This supports idealized motion profiles such as a dome and surround moving at different amplitudes.

Prescribed-velocity components use a canonical 1 m/s normal-velocity basis. Electrodynamic components use a canonical 2.83 V basis and a single rigid-body translation degree of freedom with direct Re, Le, Bl, Mmd, Cms, and Rms parameters. Their front and rear acoustic surfaces may belong to different regions and do not need matching meshes.

The electrodynamic model includes linear motor force, back EMF, mechanical mass, compliance, damping, voice-coil inductance, and reciprocal acoustic loading. It does not model cone breakup, nonlinear or position-dependent motor parameters, thermal compression, suspension nonlinearities, or flexible enclosure structures. See Coupled Solver for the precise supported contract and limitations.

Linear Excitation Bases and Channel Synthesis

The acoustic solver computes an independent complex reference response for each active excitation port. Application channels then combine those bases and apply level, polarity, delay, and idealized analog HPF/LPF transfer functions. Because this synthesis is linear and occurs after the physical solve, ordinary channel edits can reuse completed basis data without rerunning BEM or FEM.

When **Normalized Channel Correction** is enabled, Boundary Lab evaluates each channel's isolated response at the configured horizontal reference angle and applies a real magnitude correction before the channel controls. This makes the isolated channel magnitude target flat before crossover shaping. It is not a complex phase inverse, does not remove propagation or transducer phase, and does not guarantee that a summed multiway response will be flat.

Passive electrical networks, amplifier source-impedance interaction, feedback control, and level-dependent processing would alter the physical solve rather than act as ordinary post-solve channel weights. They are not part of the current application model.

Exterior Boundary Integral Model

In the unbounded region, acoustic pressure satisfies the frequency-domain Helmholtz equation:

$$\nabla^2 p + k^2 p = 0$$

where:

- `k = omega / c;`
- `omega = 2 pi f;`
- `c` is the sound speed.

The exterior mesh is the boundary of the acoustic domain. Prescribed outward normal velocity is converted to Neumann data using the solver's `exp(-i omega t)` convention:

$$q = \frac{\partial p}{\partial n} = i\rho\omega v_n$$

where `rho` is fluid density. Rigid surfaces use `v_n = 0`; moving surfaces use the component velocity projected onto the face normal. Interface surfaces in a coupled system receive their normal derivative from the FEM-BEM solution.

Boundary pressure is represented with continuous first-order (`P1`) basis functions and boundary velocity/flux with discontinuous constant (`DP0`) basis functions. BEAT Engine assembles dense Galerkin operators and uses direct dense linear algebra. The Bempp backend uses Bempp-cl operators and GMRES.

The classical exterior Neumann equation can be written as:

$$\left(K - \frac{1}{2}I\right)p = Sq$$

with single-layer operator `S`, double-layer operator `K`, identity `I`, unknown boundary pressure `p`, and prescribed Neumann data `q`. This classical form can become unreliable at fictitious interior resonances. Boundary Lab's Burton-Miller combined-field path adds the hypersingular and adjoint double-layer equations to suppress those artifacts, at additional assembly cost. The shared BEAT Engine implementation is described in BEAT Engine Core.

After solving the boundary pressure, exterior pressure at observation point `x` is evaluated from the representation formula:

$$p(x) = D[p](x) - S[q](x)$$

where `D` and `S` are the double- and single-layer potentials.

Coupled FEM-BEM Model

Each bounded region uses first-order pressure FEM on selected tetrahedral volume groups. Rigid boundaries contribute the natural zero-normal-velocity condition; moving boundaries contribute prescribed or component-coupled loads. An interface enforces pressure continuity and normal-flux conservation between its FEM boundary facets and the conforming BEM surface facets.

The current coupled model assumes linear pressure acoustics and the same density and sound speed in every participating acoustic region. Each bounded region may have its own homogeneous bulk-loss factor. Bounded rigid walls may also use the supported locally reacting, rigid-backed Miki porous treatment; this is not a thermoviscous boundary-layer model.

The coupled application path requires BEAT Engine CPU or CUDA. CPU solves the full monolithic dense coupled system. CUDA exactly eliminates eligible FEM interior degrees of freedom with a Schur complement, solves the retained interface, moving-surface, BEM, and transducer system

on the GPU, and then reconstructs the eliminated FEM pressure. These are algebraically different execution strategies for the same linear model.

Symmetry

X symmetry uses a positive-X half model; XY symmetry uses a positive-X/positive-Y quarter model. The active mesh must lie in that fundamental domain. BEAT Engine includes reflected BEM source contributions and evaluates the full observation field from the reduced solve.

Boundary Lab determines separately whether each component and FEM-BEM interface is actually cut by an active symmetry plane. For an electrodynamic component, selected moving-surface perimeter adjacency determines its surface completion factor and the number of distinct physical components represented by symmetry images. These values are inferred from topology rather than chosen from a manual multiplier.

Symmetry assumes the omitted geometry and excitation are exact mirror images. It is therefore invalid for asymmetric geometry, material properties, component parameters, or drive conditions.

SPL, Directivity, and Balloon Data

Complex pressure magnitude is converted to sound-pressure level with:

$$\mathrm{SPL} = 20\log_{10}\left(\frac{|p|}{20\times10^{-6}}\right)$$

On-axis plots and on-axis text exports retain absolute SPL for each solved channel basis after the selected channel correction. Directivity plots are normalized during visualization to their configured reference angles and then optionally smoothed and clipped to the selected display range. Polar exports contain normalized magnitude and relative phase.

When Balloon Sampling is enabled, the solver evaluates approximately equal-area Fibonacci-sphere directions. Those solved points are used directly as balloon vertices, and a shared spherical triangulation supplies the display and export topology. The balloon mesh therefore scales with the requested sampling precision without first interpolating onto a separate latitude and longitude grid. Slice plots still interpolate the spherical data along their requested great-circle directions.

Practical Limitations

Results remain subject to mesh resolution, geometry and physical-group quality, interface conformity, numerical precision, observation distance, and the assumptions above. Dense BEM storage and factorization grow rapidly with boundary size; fine meshes should be tested at a few representative frequencies before committing to a long sweep.

Boundary Lab currently does not provide:

- structural diaphragm, suspension, or enclosure-panel FEM;
- nonlinear, transient, thermal, or level-dependent behavior;

- automatic conversion from a conventional T/S set containing Mms to the direct Mmd parameter used by the electrodynamic model;
- passive-radiator components;
- arbitrary surface impedance functions or spatially varying volume loss;
- multiple unbounded exterior regions or different fluids within one coupled system;
- iterative, fast-multipole, or distributed coupled solving.

Use measurement or a suitable structural/electroacoustic tool where these effects dominate, and treat Boundary Lab's result as the prediction of the configured linear acoustic model rather than the complete behavior of a real loudspeaker.

BEAT Engine Core

Boundary Lab's BEAT Engine, short for Boundary Element Acoustic Toolkit Engine, is the Julia solver stack used for local exterior BEM and coupled FEM-BEM-LEM solves. The Python side stages mesh assets and request JSON, while `src/blab/solvers/julia_local/solver.jl` owns the numerical solve. `BeatEngineCore.jl` provides shared BEM mesh, quadrature, formulation, symmetry, Burton-Miller, and field-evaluation utilities used by both BEAT Engine CPU and BEAT Engine CUDA.

This page describes the shared exterior-BEM core and its prescribed normal velocity path. The FEM assembly, interface transfer, electrodynamic equations, and coupled block systems are documented in Coupled Solver. Local production solves use single precision (`Float32`/`ComplexF32`).

Backend-specific details live in:

- BEAT Engine CPU
- BEAT Engine CUDA

Solve Pipeline

For each solve request, Julia:

1. Loads one or more Gmsh 2.2 ASCII triangle meshes.
2. Applies per-mesh scale and translation, then combines them into one `BoundaryMesh`.
3. Builds P1 pressure and DP0 velocity spaces.
4. Builds frequency-independent identity/mass matrices.
5. For each frequency, assembles Helmholtz boundary operators.
6. Solves the Burton-Miller Neumann system for boundary pressure.
7. Evaluates SPL at polar and optional spherical observation points.
8. Computes per-radiator acoustic impedance from pressure over driven elements.

Radiators are resolved by both physical tag and mesh id, so duplicate physical tags are allowed across meshes when the radiator specifies its mesh.

Boundary Integral Formulation

The backend uses the outgoing Helmholtz Green function:

$$G_k(x, y) = \frac{e^{ik\|y - x\|}}{4\pi\|y - x\|}.$$

The assembled operator tuple contains:

- `single_layer`: maps DP0 Neumann data to P1 test functions.
- `double_layer`: maps P1 pressure data to P1 test functions.
- `adjoint_double_layer`: maps DP0 Neumann data to P1 test functions.
- `hypersingular`: maps P1 pressure data to P1 test functions.

The dense Burton-Miller system is formed as:

$$\left(\tfrac{1}{2} I_{P1,P1} - D + \eta H\right) p = \left(-S - \eta\left(D^* + \tfrac{1}{2} I_{P1,DP0}\right)\right) q,$$

where:

$$\eta = \frac{i}{k}.$$

Here (p) is the solved boundary pressure and (q) is the Neumann/radiator drive vector. Radiator normal velocity is converted to Neumann data with:

$$q = i\rho\omega v_n.$$

For a test triangle (T_i), trial triangle (T_j), test basis function (\phi_a), and trial basis function (\psi_b), a typical single-layer block entry is:

$$S_{ab}^{ij} = \int_{T_i}\int_{T_j} \phi_a(x) G_k(x, y) \psi_b(y)\, dS_y\, dS_x.$$

The double-layer and adjoint double-layer use normal derivatives of (G_k):

$$D_{ab}^{ij} = \int_{T_i}\int_{T_j} \phi_a(x) \frac{\partial G_k(x, y)}{\partial n_y} \psi_b(y)\, dS_y\, dS_x,$$

$$(D^*)_{ab}^{ij} = \int_{T_i}\int_{T_j} \phi_a(x) \frac{\partial G_k(x, y)}{\partial n_x} \psi_b(y)\, dS_y\, dS_x.$$

The hypersingular block is assembled with the surface-curl weak form:

$$H_{ab}^{ij} = \int_{T_i}\int_{T_j} G_k(x, y)\left[\operatorname{curl}_\Gamma \phi_a(x) \cdot \operatorname{curl}_\Gamma \psi_b(y) - k^2 \phi_a(x)\psi_b(y) n_x \cdot n_y\right] dS_y\, dS_x.$$

BEAT Engine stores all four dense matrices explicitly. This makes direct dense solves straightforward, but memory usage scales quadratically with the P1/DP0 unknown counts.

Singular And Regular Pairs

Operator assembly splits triangle pairs into regular pairs and adjacent/coincident singular pairs. Regular pairs are integrated directly with triangle quadrature. Adjacent, edge-sharing, vertex-sharing, and coincident pairs are handled with Duffy correction rules.

The singular path builds a frequency-independent correction cache for the mesh and singular quadrature order. The cache stores adjacent/coincident element pairs, orientation-remapped Duffy rules, surface curls, and pair geometry scalars once, then reuses them across the frequency loop. Per-frequency work still evaluates the Helmholtz kernels because they depend on (k).

The image-singular path for symmetry computes compact `singular - regular` correction blocks before adding them to the dense operators. CUDA scatters those blocks through GPU correction buffers with atomics; CPU applies the same correction logic directly on host matrices.

Symmetry Mode

BEAT Engine supports `off`, `x`, and `xy` symmetry modes in both the BEAT CUDA and BEAT CPU backends. The application disables the symmetry control for backends that do not advertise symmetry support.

The application passes a reduced-domain mesh plus symmetry metadata. BEAT Engine does not infer or match mirrored element orbits. Instead, it assumes the global origin is the symmetry origin and validates that the provided mesh lies in the positive fundamental domain:

- `x`: all mesh vertices must be on or positive of the global X=0 plane.
- `xy`: all mesh vertices must be on or positive of both the global X=0 and Y=0 planes.

For each enabled mirror plane, the backend builds image transforms. Points and normals are reflected as ordinary vectors. Surface curls in the hypersingular weak form are reflected as pseudovectors, using `det(R) * R * curl` for each mirror transform. For `x` symmetry, the reduced operator includes the identity domain plus the X-reflected image. For `xy` symmetry, it includes the identity domain plus X, Y, and XY images.

The reduced Galerkin system remains defined on the reduced mesh's P1 pressure space and DP0 Neumann space. Regular operator assembly handles symmetry by adding reflected trial/source image contributions into the reduced matrices. This keeps the solved pressure vector on the reduced P1 dofs while accounting for the missing physical images.

P1 test rows receive symmetry orbit weights for vertices on symmetry planes. These weights are applied consistently to the Helmholtz operators and to the Burton-Miller identity/mass matrices. This is required for seam vertices on X=0 and Y=0 to match the corresponding full expanded system.

Singular handling has two parts:

- Identity-domain adjacent, edge-sharing, vertex-sharing, and coincident pairs use the normal Duffy correction cache.
- Reflected image pairs that become coincident, edge-adjacent, or vertex-adjacent across a symmetry plane use an image-singular correction cache.

Field evaluation is symmetry-aware by materializing mirrored quadrature sources in the field-evaluation cache. The existing field kernels are then reused unchanged: source points and normals are reflected for each symmetry transform, while source faces, source elements, quadrature weights, and P1 basis values continue to reference the reduced mesh.

Radiator impedance is computed from reduced-domain pressure and then scaled by the symmetry reduction factor: 2 for `x`, 4 for `xy`. This reports force over the full physical radiator image set while keeping the solve unknowns reduced.

Field Evaluation

After boundary pressure is solved, the backend evaluates the potential at observation points:

$$u(x) = \int_{\Gamma} \frac{\partial G_k(x,y)}{\partial n_y} p(y) - G_k(x,y) q(y)\, dS_y.$$

The implementation precomputes a field-evaluation cache containing quadrature source points, normals, weights, source faces, source elements, and P1 basis values.

The evaluated potential is:

$$u(x) = \sum_m \left[\frac{\partial G_k(x,y_m)}{\partial n_m} p_m - G_k(x,y_m) q_m \right] w_m,$$

where (y_m), (n_m), and (w_m) come from the cached source quadrature data.

SPL is reported as:

$$\mathrm{SPL}(x) = 20 \log_{10} \left(\frac{|u(x)|}{20\ \mu\mathrm{Pa}} \right).$$

Horizontal, vertical, and spherical observation points are concatenated into one field evaluation per frequency, then sliced back into result arrays. This avoids rebuilding source strengths for each observation set.

Performance Shape

The expensive stages are:

- dense operator assembly, roughly (O(N_e^2 q^2)), where (N_e) is face count and (q) is quadrature point count
- dense direct solve, roughly (O(N_p^3)), where (N_p) is P1 dof count
- field evaluation, roughly (O(N_{\mathrm{obs}} N_e q))

Symmetry can improve runtime by more than the simple physical-area reduction would suggest because it reduces the dense system dimension. Assembly has to include image contributions, so its scaling is closer to the expected half-domain or quarter-domain work plus image passes. The direct Burton-Miller solve, however, scales roughly as (O(N_p^3)). Halving the P1 unknown count can make the dense solve approach one eighth of the full cost, and quartering it can make that portion much smaller still.

Important Files

- `src/blab/solvers/beat_engine_backend.py`: Python adapter that stages assets and streams JSON events.
- `src/blab/solvers/julia_local/solver.jl`: request handling, mesh/radiator setup, frequency loop, backend dispatch, drive calculation.
- `src/blab/solvers/julia_local/src/BeatEngineCore.jl`: mesh representation, shared quadrature/formulation code, Burton-Miller solve, field evaluation interfaces.
- `src/blab/solvers/julia_local/src/BeatEngineCpu.jl`: include hub for the CPU implementation files.
- `src/blab/solvers/julia_local/src/BeatEngineCuda.jl`: include hub for the CUDA implementation files.

BEAT Engine CPU

The BEAT Engine CPU backend is a hardware-agnostic Julia/OpenBLAS solver path. It uses the same BEAT Engine request protocol, mesh handling, Burton-Miller formulation, symmetry model, and result stream described in BEAT Engine Core, but performs operator assembly, dense solve, and field evaluation on the host.

The application exposes this path as `BEAT Engine (CPU)` / `beat_cpu`. Its coupled FEM-BEM execution path is described separately in Coupled Solver; the sections below focus on BEM operator and field work shared with exterior solves.

CPU Solve Path

For each frequency, the CPU path:

1. Selects the regular quadrature rule for that frequency.
2. Assembles dense Galerkin operators on host matrices.
3. Applies singular and image-singular Duffy corrections directly into those matrices.
4. Builds the Burton-Miller dense system.
5. Solves through Julia's dense BLAS/LAPACK path.
6. Evaluates requested polar and spherical fields on the CPU.

The CPU implementation lives under `src/blab/solvers/julia_local/src/BeatEngineCpu*.jl`.

CPU Operator Assembly

`BeatEngineCpuAssembly.jl` is the CPU Galerkin operator assembly entry point. It precomputes per-element geometry and per-element quadrature data, then loops over test/trial element pairs.

Regular pair assembly skips adjacent/coincident pairs, which are handled afterward by singular corrections. When threaded assembly is enabled and Julia has more than one thread, element coloring is used to avoid write conflicts while scattering element-block contributions into shared dense matrices.

Symmetry image contributions are assembled on the host by reflecting trial/source element geometry and quadrature data. Reflected image pairs that become singular across a symmetry plane use image-singular correction caches.

Dynamic Quadrature

The CPU path defaults to wavelength-driven regular quadrature. This is a CPU-only production feature; CUDA currently remains fixed-order as the GPU solver does not meaningfully benefit from dynamic quadrature.

The goal is to reduce low-frequency regular-pair assembly cost without changing singular-pair quadrature. Regular-pair assembly dominates dense BEM workload, and low frequencies do not need the same regular quadrature density as high frequencies on typical Boundary Lab loudspeaker meshes.

The selector computes:

$$kh = \frac{2\pi f}{c}\sqrt{A_{\mathrm{stat}}},$$

where (A_{\mathrm{stat}}) is a mesh element-area statistic. The current default statistic is `p90`, so (h = \sqrt{A_{p90}}).

Default CPU thresholds:

- q1 disabled: `wavelength_kh_q1_max = 0.0`
- q2 when `k*h <= 2.0`
- q4 above `k*h > 2.0`
- base order: `quadrature_order = 4`
- mesh statistic: `wavelength_mesh_stat = "p90"`

The selected order is cached per frequency order. The CPU path reuses per-order regular rules, identity/mass matrices, and field-evaluation caches. Singular quadrature remains controlled by `singular_order` and is not reduced by the wavelength selector.

Result diagnostics include the selected mode, selected order, mesh statistic, element length, and `k*h` value:

- `regular_quadrature_mode`
- `regular_quadrature_order`
- `regular_quadrature_base_order`
- `regular_quadrature_wavelength_mesh_stat`
- `regular_quadrature_wavelength_element_length_m`
- `regular_quadrature_wavelength_kh`
- `regular_quadrature_wavelength_kh_q1_max`
- `regular_quadrature_wavelength_kh_q2_max`

Validation Notes

The tuned defaults were validated against fixed q4 on `sample.msh` using output-level checks from `compare_cpu_quadrature.jl`.

The strongest current candidate was:

- q1 disabled
- q2 while `k*h <= 2.0`
- q4 above that
- `p90` mesh-area statistic

On `sample.msh`, this selected q2 through 4 kHz and q4 at 4.5 kHz and above. The full-mesh output comparison measured approximately:

- median operator assembly speedup: `3.26x`
- max pressure relative L2 error: `0.148%`
- max field relative L2 error: `0.442%`
- max SPL RMS delta: `0.116 dB`
- max SPL p95 delta: `0.265 dB`
- max SPL max delta: `0.356 dB`

Raw operator relative error can exceed 1% even when solved pressure and field outputs remain well below the intended output-error target. Threshold tuning should therefore prioritize solved/output metrics over operator norms alone.

CPU Field Evaluation

`BeatEngineCpuField.jl` evaluates the same field integral described in BEAT Engine Core. It uses the CPU field-evaluation cache for the selected quadrature order and computes the

potential at concatenated horizontal, vertical, and optional spherical observation points.

Field evaluation is usually not the dominant CPU cost compared with dense operator assembly and dense solve.

CPU Dense Solve

`BeatEngineCpuSolve.jl` forms the Burton-Miller system on host matrices and calls Julia's dense solve path. Runtime depends heavily on BLAS/LAPACK performance and P1 unknown count. Symmetry can reduce solve cost significantly because dense solve complexity scales roughly as (O(N_p^3)).

The CPU backend selects the BLAS thread count from the P1 unknown count:

- up to 768 P1 unknowns: 1 BLAS thread
- 769 to 2,048: up to 4 BLAS threads
- 2,049 to 4,096: up to 8 BLAS threads
- above 4,096: all available Julia threads

Set `BLAB_BEAT_CPU_BLAS_THREADS` to `auto` or a positive integer to override the policy. Explicit values are capped at the Julia thread count.

CPU Benchmark And Comparison Scripts

Useful scripts:

- `src/blab/solvers/julia_local/scripts/benchmark_cpu.jl`: CPU timing benchmark, including fixed and wavelength regular quadrature modes.
- `src/blab/solvers/julia_local/scripts/benchmark_cpu_blas.jl`: synthetic or real-system dense LU thread-scaling benchmark.
- `src/blab/solvers/julia_local/scripts/compare_cpu_quadrature.jl`: fixed-reference versus candidate comparison artifact generator with operator, pressure, field, and SPL error metrics.

Example comparison:

```
& 'C:\Users\John\AppData\Local\Programs\Julia-1.12.6\bin\julia.exe' `
  src\blab\solvers\julia_local\scripts\compare_cpu_quadrature.jl `
  --frequencies 20,50,100,200,500,1000,1500,2000,3000,4000,4500,5000 `
  --subset-faces 0 `
  --output-points 72 `
  --wavelength-kh-q1-max 0 `
  --wavelength-kh-q2-max 2.0 `
  --json src\blab\solvers\julia_local\results\cpu_quadrature_compare_outpu
```

Important Files

- `src/blab/solvers/julia_local/src/BeatEngineCpu.jl`: include hub for the

CPU implementation files.

- `src/blab/solvers/julia_local/src/BeatEngineCpuAssembly.jl`: CPU Galerkin operator assembly entry point.
- `src/blab/solvers/julia_local/src/BeatEngineCpuField.jl`: CPU field-evaluation path.
- `src/blab/solvers/julia_local/src/BeatEngineCpuSolve.jl`: CPU Burton-Miller dense solve through Julia's LAPACK/BLAS path.
- `src/blab/solvers/julia_local/solver.jl`: CPU backend dispatch, wavelength quadrature selection, per-order caches, and result diagnostics.

BEAT Engine CUDA

The BEAT Engine CUDA backend is an NVIDIA GPU-accelerated Julia solver path. It uses the same BEAT Engine request protocol, mesh handling, Burton-Miller formulation, symmetry model, and result stream described in BEAT Engine Core, but performs regular-pair assembly, singular corrections, dense solve, and field evaluation with CUDA.jl.

The application exposes this path as `BEAT Engine (CUDA)` / `beat_cuda`. Its coupled FEM-BEM execution path, including FEM static condensation, is described separately in Coupled Solver; the sections below focus on BEM operator and field work shared with exterior solves.

CUDA Operator Assembly

The CUDA path assembles dense Galerkin operators by splitting triangle pairs into regular pairs and adjacent/coincident singular pairs. Regular pairs are assembled by CUDA kernels over test/trial element pairs. Adjacent, edge-sharing, vertex-sharing, and coincident pairs are handled by a separate GPU Duffy correction path.

The application backend requires CUDA and moves geometry and quadrature arrays to the GPU:

- face vertices
- normals
- areas
- global face indices
- surface curls
- quadrature points and weights
- test/trial element index arrays

`build_cuda_regular_assembly_cache` keeps these arrays resident across frequencies, avoiding repeated host-to-device transfers for fixed mesh geometry.

For each frequency, CUDA allocates real and imaginary dense matrices for:

- SLP real/imag
- DLP real/imag
- adjoint DLP real/imag
- hypersingular real/imag

The regular-pair kernel skips adjacent pairs. Singular and near-singular corrections use Duffy quadrature and are added after regular assembly.

CUDA Singular Corrections

A Duffy block kernel computes compact per-pair correction blocks directly on device. A CUDA scatter kernel atomically accumulates those compact blocks into dense correction buffers before adding them to the resident operators. The Duffy kernel reuses the regular assembly geometry cache, so the per-frequency singular correction path does not transfer dense CPU correction matrices. Symmetry image-singular topology is also mirrored to a device cache once during job initialization and reused across the frequency sweep.

The CUDA singular correction cache stores:

- test and trial element indices
- rule indices for orientation-remapped Duffy rules
- P1 row/column dofs and DP0 columns
- pair Jacobian scales and normal products
- flattened remapped Duffy points and weights

Symmetry image-singular pairs use the same compact correction idea, with reflected image geometry and GPU scatter.

CUDA Kernel Modes

The CUDA backend has a regular assembly split kernel, one singular correction block kernel, and two field-evaluation kernels.

Regular assembly uses a serial pair batched split path. A CUDA block carries 128 independent regular test/trial element pairs, with one CUDA thread responsible for the full quadrature loop for one pair. For the current order-4 regular triangle rule, each thread evaluates the 36 quadrature-point pairs for its element pair directly in registers and then atomically scatters the resulting element-block entries.

The regular path uses two regular assembly launches:

- `_cuda_regular_quadrature_slp_hyp_kernel!` computes single-layer and hypersingular contributions.
- `_cuda_regular_quadrature_dlp_adjoint_kernel!` computes double-layer and adjoint double-layer contributions.

This grouping keeps both launches at 24 accumulator slots, which reduces register/shared-memory pressure compared to a fused all-operator kernel.

The serial pair batched split kernels atomically scatter real and imaginary element-block entries into dense operator buffers. Singular adjacent/coincident pairs are skipped during regular assembly and handled afterward by the Duffy correction path.

The split regular launches use an 80-register compiler cap. On the reference RTX 2080 Ti this raises achieved occupancy from about 49% to 73% while keeping compiler spill traffic predominantly L2-resident. The cap was selected by comparing unrestricted, 88, 84, 80, and 72-register variants on `sample_detailed.msh`; lower caps lost performance to spill traffic.

`_cuda_duffy_blocks_kernel!` maps GPU threads over cached adjacent/coincident element pairs. Each thread computes the compact singular correction block for one or more pairs using the cached remapped Duffy rule. `_cuda_singular_scatter_kernel!` then atomically scatters those compact blocks into dense GPU correction buffers.

`_cuda_weighted_field_sources_kernel!` maps GPU threads over cached source quadrature points and builds weighted pressure and Neumann source strengths. `_cuda_field_eval_kernel!` maps one CUDA block to one observation point and reduces source contributions in dynamic shared memory.

CUDA Atomics

The GPU kernels accumulate element-block contributions into global dense matrices. Multiple CUDA blocks can target the same global row/column entries, especially because P1 basis functions are shared across adjacent faces. The regular assembly and singular scatter paths use device atomics for global accumulation:

```
@inline function _cuda_atomic_add!(array, index, value)
    CUDA.@atomic array[index] += value
    return nothing
end
```

Dense operator accumulation buffers are stored as separate real and imaginary arrays during kernel execution, so atomic additions operate on scalar `Float32` values. After the kernel finishes, real and imaginary matrices are materialized into complex matrices on GPU:

$$A = A_{\mathrm{re}} + iA_{\mathrm{im}}.$$

This avoids a slow serial scatter stage and lets regular-pair assembly remain massively parallel. The serial pair batched regular assembly path preserves this property: it atomically accumulates into the same dense buffers through two balanced operator-family kernels instead of one all-operator kernel. The tradeoff is that floating-point atomic accumulation is order-dependent, so tiny run-to-run differences can occur at the last few bits.

GPU Dense Solve

If operators are returned on GPU, `solve_burton_miller_neumann` forms the Burton-Miller

matrix and RHS directly as `CuArray`s:

$$A = \frac{1}{2}I - D + \eta H,$$

$$b = \left(-S - \eta(D^* + \frac{1}{2}I_{P1,\,DP0})\right)q.$$

For systems above 768 P1 unknowns, the RHS is evaluated matrix-free with three accumulating GPU matrix-vector products, avoiding a dense combined RHS-operator temporary. Smaller systems retain the single-GEMV materialized path because it benchmarks faster. The solver then calls Julia's dense linear solve:

```
d_pressure = d_lhs \ d_rhs
```

With CUDA arrays, this dispatches through CUDA.jl to GPU dense linear algebra. The solved pressure vector is currently copied back to CPU after the solve; CUDA field evaluation then transfers pressure and Neumann vectors back to the GPU for the observation pass. This keeps the public result path simple while leaving room to keep pressure resident across downstream stages later.

Temporary GPU allocations are explicitly released with `CUDA.unsafe_free!` after assembly or solve stages to reduce memory pressure during frequency sweeps.

CUDA Field Evaluation

`build_cuda_field_evaluation_cache` mirrors the shared field-evaluation cache to GPU as a `CudaFieldEvaluationCache`, using structure-of-arrays storage for coalesced source-point and normal reads.

For each frequency, CUDA field evaluation uses two stages:

1. `_cuda_weighted_field_sources_kernel!` interpolates solved P1 pressure to every source quadrature point, multiplies by quadrature weights, and builds the weighted Neumann source term.
2. `_cuda_field_eval_kernel!` assigns one CUDA block to each observation point. Threads in the block stride over all source quadrature points, evaluate the single-layer and double-layer kernels, accumulate local real/imaginary potentials, and reduce those partial sums in dynamic shared memory.

The result is materialized as a compact potential vector and copied back for SPL conversion and result serialization.

Quadrature Mode

CUDA currently uses fixed regular quadrature order for the frequency sweep. Wavelength-driven regular quadrature is implemented only for the BEAT CPU path at this time.

CUDA's regular assembly cache is built from `mesh + rule`, so future CUDA wavelength quadrature would need per-order CUDA regular caches and corresponding field/identity cache

handling.

Performance Notes

CUDA accelerates regular-pair assembly, singular Duffy corrections, the dense solve, and field evaluation. The remaining dominant cost in CUDA solves is usually regular-pair assembly for the dense operators, followed by the dense solve for larger P1 systems. The serial pair batched regular assembly mode reduces regular-kernel pressure by assembling SLP/hypersingular and DLP/adjoint in separate launches while keeping dense operators resident on the GPU and batching 128 regular pairs per CUDA block.

Temporary allocation, dense GPU memory footprint, and atomic accumulation cost are important practical limits. CUDA memory use scales with dense P1/DP0 matrix dimensions, so symmetry is especially useful on large meshes.

`scripts/benchmark_cuda.jl` measures the serial pair batched regular assembly path:

```
julia scripts\benchmark_cuda.jl --skip-solve --skip-field --warmups 3 --re
```

Use `sample_detailed.msh` with multiple warmups for hardware comparisons. Nsight Compute can profile the measured regular kernels with a kernel filter such as `regex:.*regular_quadrature.*`; on Windows, detailed counters require NVIDIA performance-counter permission to avoid `ERR_NVGPUCTRPERM`.

Important Files

- `src/blab/solvers/julia_local/src/BeatEngineCuda.jl`: include hub for the CUDA implementation files.
- `src/blab/solvers/julia_local/src/BeatEngineCudaCommon.jl`: CUDA package setup, shared cache structs, and shared device helpers.
- `src/blab/solvers/julia_local/src/BeatEngineCudaRegular.jl`: CUDA geometry/rule cache builders and regular-pair kernels.
- `src/blab/solvers/julia_local/src/BeatEngineCudaSingular.jl`: GPU Duffy corrections, singular cache mirroring, image-singular corrections, and scatter kernels.
- `src/blab/solvers/julia_local/src/BeatEngineCudaOperators.jl`: GPU operator storage helpers, timing helpers, and regular kernel launch helpers.
- `src/blab/solvers/julia_local/src/BeatEngineCudaAssembly.jl`: public CUDA Galerkin operator assembly entry point.
- `src/blab/solvers/julia_local/src/BeatEngineCudaField.jl`: GPU field-evaluation cache, source weighting, and observation kernels.
- `src/blab/solvers/julia_local/src/BeatEngineCudaProfiling.jl`: optional CUDA regular-kernel probe and profiling launches used by benchmark scripts.

Advanced CLI Workflow

Boundary Lab still includes command-line tools for mesh cleaning, solving, data preparation, and static plot generation. The GUI is the recommended entry point for normal use.

The CLI workflow is:

1. `blab clean`
2. `blab solve`
3. `blab prepare`
4. `blab plot`

Clean A Mesh

```
blab clean input.msh output_clean.msh --merge-tol 1e-9
```

This merges coincident vertices, removes degenerate or duplicate triangles, and writes Gmsh 2.2 format.

Run A Solve

```
blab solve output_clean.msh --output-npz pressure_data_raw.npz --freq-min
```

Useful options include:

- `--config`
- `--output-npz`
- `--freq-min`
- `--freq-max`
- `--freq-count`
- `--step-size`
- `--min-angle`
- `--max-angle`
- `--axial-offset`
- `--workers`
- `--gmres-tol`
- `--spherical-sampling`
- `--spherical-sampling-points`

The legacy solve CLI accepts a TOML file for multi-mesh, multi-radiator exterior-BEM jobs. Paths are resolved relative to the TOML file. For example:

```
[[meshes]]
name = "cabinet"
file = "cabinet.msh"
scale_factor = 0.001
```

```
translation_m = [0.0, 0.0, 0.0]

[[radiators]]
name = "woofer"
mesh = "cabinet"
tag = 2
channel = "main"
velocity_offset_db = 0.0

[radiators.hpf]
filter = "butterworth"
order = 2
frequency_hz = 80.0
```

Each radiator requires `name` and integer physical `tag`; `mesh` is required when more than one configured mesh could contain that tag. Optional radiator fields are `channel`, `velocity_offset_db`, `level_db`, `polarity`, `delay_ms`, `hpf`, and `lpf`.

Prepare Visualization Data

```
blab prepare pressure_data_raw.npz pressure_data_formatted.npz --min-db -3
```

This applies clipping, interpolation, normalization, and smoothing for the static plot pipeline.

Generate Static Plots

```
blab plot pressure_data_formatted.npz --output-dir .
```

This writes:

- `horizontal_isobar.png`
- `vertical_isobar.png`
- `acoustic_impedance.png`

Notes

The CLI path is useful for scripted exterior-BEM workflows and regression checks. It does not load `.blab.json` physical systems, run coupled FEM-BEM models, or expose the GUI's live plot behavior.

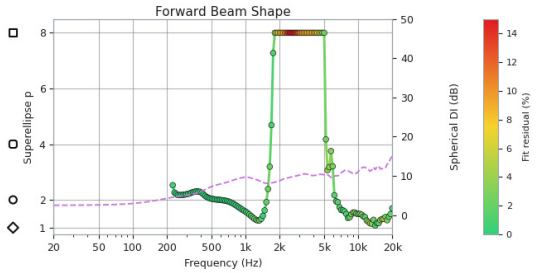
Forward Beam Shape Plot

The Forward Beam Shape plot is a diagnostic view in the balloon plot window. It reduces each front-facing spherical radiation balloon to two frequency-dependent traces:

- the fitted superellipse exponent `p` of the forward `-6 dB` beam contour
- the spherical directivity index in dB

The goal is to describe how the forward beam shape changes with frequency. It is not a replacement for the full balloon, isobar, or polar views. It is a compact shape descriptor that makes broad trends easier to see, especially for horns, waveguides, and other non-axisymmetric

radiators.



What The Plot Shows

The left axis shows the fitted superellipse exponent `p` for the front-facing `-6 dB` contour:

- `p = 1`: diamond or rhombus-like contour
- `p = 2`: ellipse or circle-like contour
- `p = 4`: rounded square-like contour
- `p = 8`: squarer contour with harder corners

The plotted points are colored by fit residual. Green means the extracted contour is well-described by the nearest superellipse. Red means the contour is less superellipse-like, usually because it has lobing, ripples, asymmetry, notches, or local diffraction features.

The right axis shows `Spherical DI (dB)`, computed from the full spherical SPL data at each frequency. This lets the beam shape trend be compared against the overall concentration of radiated energy.

A vertical cursor line follows the current frequency selected in the balloon viewer.

Source Data

The plot uses the solved Fibonacci balloon samples directly:

- `freq_hz`: solved frequencies
- `theta_polar_rad`: polar angle from the forward `+z` axis
- `phi_azimuth_rad`: azimuth angle around `+z`
- `directions_xyz`: unit direction for every solved sample
- `triangle_indices`: shared triangular surface topology
- `balloon_surface_spl`: normalized SPL in dB at the solved vertices

The balloon coordinate convention is:

```
x = horizontal right
y = vertical-side axis
z = forward reference axis
```

For each spherical sample, the corresponding unit direction is:

$$x = \sin\theta\cos\phi,$$
$$y = \sin\theta\sin\phi,$$
$$z = \cos\theta.$$

Only the forward hemisphere is used for the beam shape fit. Boundary Lab also limits the front angular map to approximately `+/-89 deg` horizontally and vertically so the tangent-plane transform remains finite.

Front Tangent-Plane Coordinates

The shape fit is performed in a tangent front plane rather than directly in independent horizontal and vertical angle coordinates. This is important for wide beams.

The horizontal and vertical angular coordinates are first computed as:

$$\alpha = \operatorname{atan2}(x, z),$$
$$\beta = \operatorname{atan2}(y, z).$$

They are then mapped to tangent-plane coordinates:

$$u = \tan(\alpha),$$
$$v = \tan(\beta).$$

This is equivalent to projecting the forward spherical data onto the plane tangent to the front axis. A circular cone on the sphere remains circular in this coordinate system. That avoids an artifact where very wide axisymmetric beams can look artificially square when fitted in raw horizontal/vertical angle space.

Boundary Lab builds a 2D interpolator from the forward samples:

$$S_f(u, v),$$

where `S_f` is normalized SPL in dB for one frequency `f`.

Extracting The -6 dB Contour

For each frequency, Boundary Lab samples radial rays in the tangent plane:

$$(u(r), v(r)) = (r\cos\psi, r\sin\psi),$$

where `psi` is the ray angle around the forward axis.

Along each ray, the app finds the first crossing where the interpolated SPL falls through the target level:

$$S_f(u(r), v(r)) = -6 \text{ dB}.$$

The crossing radius is linearly interpolated between adjacent ray samples. This produces a set of contour samples:

$$(\psi_i, r_i).$$

Frequencies are considered invalid for the shape trace when there is not enough usable `-6 dB` contour data. This can happen at low frequencies when the forward hemisphere has not yet fallen to `-6 dB`.

Aspect-Corrected Superellipse Fit

The fitted shape family is a superellipse in polar form. For horizontal radius `a`, vertical radius `b`, and exponent `p`, the model radius is:

$$r_{model}(\psi) = \left(\left| \frac{\cos\psi}{a} \right|^p + \left| \frac{\sin\psi}{b} \right|^p \right)^{-1/p}.$$

Boundary Lab estimates the horizontal and vertical extents from the `-6 dB` crossings along the horizontal and vertical tangent-plane axes. These extents become the aspect correction terms `a` and `b`.

The exponent `p` is then found by minimizing mean squared radial error:

$$\operatorname{argmin}_p \frac{1}{N} \sum_i (r_i - r_{model}(\psi_i))^2.$$

The current fit bounds are approximately:

```
0.75 <= p <= 8.0
```

The displayed beamwidth values remain angular quantities. Tangent-plane extents are converted back to degrees with:

$$\theta_{deg} = \frac{180}{\pi} \tan^{-1}(r).$$

For example, the reported horizontal beamwidth is the sum of the positive and negative horizontal crossing angles.

Fit Residual

The fit residual is a normalized RMS radial error in the tangent plane:

$$\text{residual} = 100 \frac{\sqrt{\frac{1}{N} \sum_i (r_i - r_{model}(\psi_i))^2}}{\frac{a+b}{2}}.$$

The residual is shown as the color of the `p` trace. The color scale is fixed from `0` to `15` percent so plots can be compared across projects.

Low residual means the contour is close to the fitted superellipse. High residual means the

contour is not well-described by this four-way shape family, even if the fitted `p` value is still plotted.

Spherical Directivity Index

The secondary axis shows spherical directivity index computed from the spherical normalized SPL data.

Boundary Lab uses the equal-area Fibonacci samples directly. Since SPL has already been normalized to the reference axis, it converts normalized dB values to relative linear energy:

$$E_i = 10^{S_i/10}.$$

The mean relative energy over the sphere is:

$$\bar{E} = \frac{1}{N}\sum_i E_i.$$

The spherical directivity index is then:

$$DI = -10\log_{10}(\bar{E}).$$

Interpreting Results

A flat trace near `p = 2` usually indicates an axisymmetric or ellipse-like forward beam. A trace moving upward toward `p = 4` or `p = 8` indicates a squarer contour. A trace moving downward toward `p = 1` indicates a more diamond-like contour.

The exponent should be read together with residual color:

- low residual and stable `p`: the contour is well-described by the shape family
- high residual and unstable `p`: the contour likely has lobing or local features that a superellipse cannot summarize well
- invalid low-frequency points: the forward `-6 dB` contour may not exist inside the front map

The Spherical DI trace helps distinguish a shape change from a simple narrowing or widening of the beam. For example, `p` may remain close to `2` while DI rises, meaning the beam is becoming narrower but staying circular.

Limitations

The Forward Beam Shape plot intentionally compresses a 3D radiation pattern into a few scalar values per frequency. It does not show:

- rear radiation
- side lobes outside the forward contour

- multiple disconnected `-6 dB` regions
- asymmetry not captured by a single horizontal and vertical aspect ratio
- local ripple around the contour, except indirectly through residual

Use the plot as a trend view. When the residual rises, inspect the full balloon, contour lines, polar slices, and isobar views to understand the actual radiation pattern.

Boundary Lab Server

Boundary Lab can run a local or LAN-accessible solve server. The GUI submits a complete solve request over HTTP, uploads the required mesh assets with the job, and streams per-frequency results back as newline-delimited JSON events.

Starting The Server

Run the server with an explicit solver selector:

```
blab server --host 127.0.0.1 --port 8765 --solver bempp_cpu
blab server --host 127.0.0.1 --port 8765 --solver beat_cpu --julia-threads
blab server --host 127.0.0.1 --port 8765 --solver beat_cuda --julia-thread
```

For LAN use, bind to the machine's LAN address or to all interfaces:

```
blab server --host 0.0.0.0 --port 8765 --solver beat_cuda
```

Then use `http://<server-ip>:8765` as the GUI's Solve Server URL.

Optional Authentication

Authentication is opt-in. If `BLAB_AUTH_TOKEN` is unset or empty, the server continues to accept unauthenticated requests for localhost and trusted private network use.

To protect a server, set a high-entropy token in the environment before starting it:

```
export BLAB_AUTH_TOKEN="paste-generated-token-here"
blab server --host 0.0.0.0 --port 8765 --solver beat_cuda
```

When a token is configured, every API endpoint, including `/health`, requires an `Authorization: Bearer ...` header. The server never logs the token.

In Boundary Lab, select `BEM Solver: Server` in Preferences, click `Generate` beside `Server access token`, and use `Copy` to place the generated token in your deployment's secret store. Keep a safe copy of the token: Boundary Lab retains it only for the current application session and does not store it in the operating system credential vault or application settings.

Bearer tokens must be used with HTTPS except for loopback addresses. Boundary Lab refuses to send a configured token over remote plain HTTP. An unauthenticated server can still use plain HTTP on a trusted private network.

Solver Selectors

Supported `--solver` values:

- `bempp_cpu`: Bempp OpenCL CPU backend.
- `beat_cpu`: BEAT Engine CPU backend through Julia.
- `beat_cuda`: BEAT Engine CUDA backend through Julia.
- `beat_rocm`: BEAT Engine ROCm selector. This is accepted by the server CLI, but the ROCm implementation is currently a placeholder and reports not implemented.

BEAT Engine server options are intentionally narrow:

- `--julia-executable`: Julia executable path. Defaults to `julia`.
- `--julia-threads`: Julia thread count. Defaults to `auto`.

The server defaults to `--solver bempp_cpu` when no solver is specified.

GUI Workflow

In the Boundary Lab GUI:

1. Open `Edit > Preferences`.
2. Set `BEM Solver` to `Server`.
3. Set `Solve Server URL` to the server address, such as `http://127.0.0.1:8765`.
4. Click `Check Server`.
5. Confirm the server info dialog, then accept Preferences.

`Check Server` calls `GET /health` with the configured access token and updates the application's view of server-advertised capabilities. This matters for features such as BEAT Engine server-side X/XY symmetry. If Boundary Lab starts with `BEM Solver` already set to `Server`, it also runs a silent startup `GET /health` probe with a 5 second timeout. Failed startup probes do not interrupt application launch; the GUI simply falls back to the conservative unavailable state until a later successful check.

Symmetry Support

Server-side symmetry depends on the configured server solver:

- `beat_cpu` and `beat_cuda` advertise symmetry support and can solve `off`, `x`, and `xy` symmetry requests.
- `bempp_cpu` does not support symmetry acceleration.
- `beat_rocm` advertises the BEAT Engine shape but is not numerically implemented yet.

The GUI uses the checked server health payload to decide whether the symmetry control in the **Meshes** window is enabled while the selected BEM Solver is `Server`.

For symmetry solves, the GUI still prepares and uploads the reduced-domain mesh files. The

server does not need access to the client's original local paths.

Job API

The server exposes a small HTTP API:

- `GET /health`: returns status, configured solver, backing backend ID, and capability flags.
- `POST /jobs`: submits a solve request with `SimulationConfig`, `frequencies_hz`, and optional uploaded assets.
- `GET /jobs/{job_id}`: returns job status and artifact links.
- `GET /jobs/{job_id}/events?since=0`: streams job events as newline-delimited JSON.
- `POST /jobs/{job_id}/cancel`: requests cancellation.
- `GET /jobs/{job_id}/artifacts/result.npz`: downloads the completed result bundle.

Typical event flow:

```
queued
started
initialized
result
result
...
completed
```

Failures are emitted as `failed` events with an error message. Cancellation is cooperative and may wait for the current in-flight frequency solve to finish.

Each event-stream response is intentionally limited to 25 seconds by default. The client reconnects with the last received event index, so long solves remain compatible with HTTPS proxies that cap connection duration without duplicating results.

Artifacts

Completed jobs write a compressed `result.npz` under the configured artifact directory. By default this is:

```
runs/server_jobs
```

Override it with:

```
blab server --artifact-dir runs/my_server_jobs --solver beat_cpu
```

Terminal job state and artifacts are retained for 24 hours by default. Expired jobs are purged during subsequent health checks or job submissions. Set `--job-retention-hours 0` to retain them indefinitely.

Resource Limits

Servers apply these defaults:

Limit	Default	Option
JSON request body	20 MiB	`--max-request-mb`
Decoded uploaded assets per job	14 MiB	`--max-asset-mb`
Frequencies per job	1000	`--max-frequencies`
Waiting jobs	4	`--max-queued-jobs`
Terminal job retention	24 hours	`--job-retention-hours`
Event response window	25 seconds	`--event-stream-window-seconds`

The 14 MiB decoded-asset limit leaves space for base64 expansion and request metadata inside the 20 MiB request ceiling. These limits are configurable for unusually large meshes.

Operational Notes

- Keep `--max-running-jobs 1` unless you have intentionally tested concurrent jobs for the selected backend and hardware.
- A server listening on a non-loopback address without `BLAB_AUTH_TOKEN` logs a warning but continues to start.
- BEAT Engine CUDA jobs should usually be run one at a time per GPU.
- Use `--log-level INFO` for normal pod logs, or `--log-level DEBUG` when diagnosing request and job flow.
- The GUI uploads mesh assets with every server job, so the server can run on another machine without shared filesystem paths.
- If a BEAT Engine server fails during startup, check that the matching Julia environment has been instantiated.

Install examples:

```
julia --project=src/blab/solvers/julia_local -e "using Pkg; Pkg.instantiat
julia --project=src/blab/solvers/julia_cuda -e "using Pkg; Pkg.instantiate
```

Coupled Solver

Boundary Lab's coupled solver models one or more bounded air volumes and the surrounding unbounded air in one frequency-domain acoustic solve. Each bounded region is solved with tetrahedral finite elements (FEM), the exterior is solved with the BEAT Engine boundary element method (BEM), and conforming port interfaces transfer pressure and normal derivative between them.

The application infers this path when a physical system contains one or more bounded regions. A system containing only its unbounded exterior instead uses the established exterior-BEM solver.

For the concepts used in the System window, see Physical System Model. For mesh and project-file details, see Inputs and Outputs.

Production paths at a glance

Coupled application solves require **BEAT Engine (CPU)** or **BEAT Engine (CUDA)**. Both use `Float32/ComplexF32`, solve all configured excitation ports as independent reference bases, and stream one result per frequency.

	BEAT Engine CPU	**BEAT Engine CUDA**
FEM matrices	Sparse assembly on CPU	Sparse assembly on CPU, copied to GPU
BEM operators	Assembled on CPU	Assembled on GPU
Coupled system	Full monolithic dense system on CPU	Schur-condensed acoustic/ electromechanical system on GPU
Factorization	CPU dense LU	cuDSS plus GPU dense LU when condensed; GPU dense LU when monolithic
Exterior field	Evaluated on CPU	Evaluated on GPU
Default Julia threads	8	4

CUDA therefore accelerates more than BEM assembly. FEM volume-interior unknowns are eliminated with an exact Schur complement, the reduced coupled matrix is assembled and factored on the GPU, and eliminated FEM pressure is reconstructed after the coupled solve. Electrodynamic models retain the FEM nodes on their diaphragm surfaces in addition to the port-interface nodes, so the condensed coupling remains exact.

A separate backend-only **reference path** uses `Float64/ComplexF64` on the CPU. It retains the full monolithic matrix and enables additional residual and BEM-replay checks. It exists for correctness testing and is not selectable as an application solver.

Model requirements

The production backend currently accepts a deliberately focused physical system:

- one or more bounded-air regions, currently with one FEM mesh each;
- exactly one unbounded-air region with one BEM mesh;
- zero or more FEM-BEM interfaces into that exterior mesh;

- one or more selected physical volume groups in each bounded region;
- ideal prescribed-velocity components, linear electrodynamic transducers, or both;
- one `normal_velocity` port per ideal source or one `voltage` port per electrodynamic transducer;
- single-axis rigid-body piston motion with an explicit `motion_axis`;
- one or more FEM or BEM moving boundaries per electrodynamic transducer;
- direct `Re`, `Le`, `Bl`, `Mmd`, `Cms`, and `Rms` transducer parameters;
- rigid boundaries everywhere else except configured interface boundaries;
- linear pressure acoustics, with optional homogeneous bulk loss in every bounded FEM air volume;
- `off`, `x`, or `xy` symmetry, with explicit completion/orbit semantics for electrodynamic components.

The FEM input must be a Gmsh 4.1 ASCII mesh containing first-order tetrahedra and triangular boundary facets. The BEM input must be a Gmsh 2.2 ASCII mesh containing first-order triangles. Boundary Lab applies each mesh's scale and translation before checking the interface and solving.

Every tagged surface belonging to an active region must have one boundary assignment. The System editor offers only `rigid`, `moving`, and `interface`; new surfaces and legacy `unused` assignments default to `rigid`. The coupled backend supports those same three assignments. A source used by an ideal prescribed-velocity component must act on a moving FEM boundary. An electrodynamic component may couple the same rigid-body degree of freedom to moving boundaries in several FEM regions, as well as to BEM moving boundaries. The independently meshed front and rear diaphragm surfaces do not need a node-to-node map because they communicate through the shared mechanical degree of freedom.

A sealed enclosure therefore has no FEM-BEM acoustic interface: its rear diaphragm is a moving FEM boundary, its front diaphragm is a moving BEM boundary, and both belong to the same electrodynamic component. The interior and exterior acoustic domains communicate only through that component's mechanical degree of freedom. A ported enclosure adds a conforming FEM-BEM interface at each port mouth; its diaphragm boundaries remain moving boundaries rather than interface boundaries.

Current medium-property rule

The current backend requires the same sound speed and density in every bounded and unbounded acoustic region. The shared sound speed sets the FEM and BEM wavenumber, and the shared density converts velocity to pressure normal derivative. Differing fluids are rejected.

Setting up a coupled solve

1. Import the tetrahedral FEM mesh or meshes and the triangular BEM mesh. Confirm their

scale, translation, and physical-group names.

2. In **System > Regions**, create a bounded-air region for each FEM chamber and one unbounded-air region. Select each active FEM volume group.
3. In **Boundaries**, classify every active physical surface. Use `moving` for prescribed FEM radiators, `interface` for both sides of the opening, and `rigid` for the remaining walls.
4. In **Interfaces**, use **Build/Identify Interfaces** to create and validate every FEM-to-BEM port connection.
5. In **Components**, attach prescribed-velocity or electrodynamic components to the appropriate moving boundaries, enter any relative surface-velocity offsets and transducer parameters, and assign application channels.
6. Set **FEM Bulk Loss Factor** on any bounded region that requires volume damping, and optionally configure **Wall Impedance** on bounded rigid surfaces. Select **BEAT Engine (CPU)** or **BEAT Engine (CUDA)** in Preferences, then choose full, X-half, or XY-quarter symmetry in **Meshes**.
7. Run the normal application solve.

The component editor accepts direct Re, Le, Bl, Mmd, Cms, and Rms values for an electrodynamic transducer, plus an automatic or manual motion axis. Component symmetry completion and physical-driver count are inferred from moving-surface adjacency to the active symmetry planes rather than selected manually.

The physical-system compiler resolves group names to Gmsh tags, checks boundary coverage and model relationships, and records explicit interface vertex, face, and orientation maps before Julia starts. Unsupported boundary roles and component models are rejected rather than silently substituted.

Interface requirements

The FEM and BEM sides of the interface must have:

- the same vertex coordinates within the configured tolerance;
- the same triangle connectivity;
- a one-to-one vertex and face correspondence;
- FEM triangles that are actual boundary facets of the selected tetrahedra;
- a recorded sign for the relative FEM and BEM face-normal orientation.

The FEM boundary facets are authoritative. If an imported BEM interface does not conform, **Build/Identify Interfaces** writes a derived Gmsh 2.2 BEM mesh and stores that derived mesh in project state. It does not modify the original mesh or retetrahedralize the FEM volume.

Boundary Lab also monitors enabled imported source meshes when the application regains focus. If either side of a configured interface changes, it verifies the dependency and rebuilds the derived conforming BEM mesh when required.

There are two conformance cases:

- If the FEM and BEM interface seams already have matching discrete vertices and edges, the interface may be fully curved. Boundary Lab replaces the BEM interface directly and leaves its surrounding surface unchanged.
- Otherwise, the two seams must describe the same planar opening. Boundary Lab copies the FEM interface facets and remeshes the surrounding planar BEM surface to meet their perimeter. That surrounding surface may span multiple Gmsh geometrical entities.

When several openings occupy the same planar BEM surface, interface construction protects the other tagged openings while rebuilding each pair. This allows, for example, independently meshed front- and rear-chamber ports to connect to one exterior boundary without consuming one another's physical groups.

Without symmetry, the completed BEM surface must be watertight. With X or XY symmetry, open BEM boundary edges are allowed only on the active symmetry planes.

Numerical formulation

FEM regions

Each bounded region uses continuous first-order pressure basis functions on tetrahedra. The disconnected FEM meshes are assembled into a block-diagonal aggregate pressure system. Boundary Lab assembles analytic stiffness and consistent mass matrices once for the frequency sweep. At angular frequency (\omega=2\pi f), the FEM Helmholtz matrix is

$$A_F = K - k^2 M - ik^2 M_\eta, \qquad k = \frac{\omega}{c},$$

where (M_\eta) is the consistent FEM mass matrix weighted by the loss factor of each bounded region. **FEM Bulk Loss Factor** is configured per region in the System window using the presets `0`, `0.002`, `0.005`, `0.01`, `0.02`, and `0.05`. `0` reproduces the lossless formulation. With the solver's (\exp(-i\omega t)) convention, the negative imaginary mass term is passive. Near an isolated lightly damped cavity mode, (Q) is approximately (1/\eta).

This remains a deliberately simple frequency-independent volume-loss approximation. It does not damp the exterior BEM domain or reproduce frequency-dependent thermoviscous boundary-layer losses. The region values used by a solve are included in its result diagnostics.

Rigid surfaces of a bounded region may also carry a rigid-backed porous wall treatment. For layer thickness (d), airflow resistivity (\sigma), and (X=f/\sigma), Boundary Lab evaluates the Miki characteristic impedance and complex wavenumber,

$$\frac{Z_c}{\rho c} = 1 + 0.070X^{-0.632} - i\,0.107X^{-0.632}, \qquad \frac{k_c}{k} = 1 + 0.109X^{-0.618} - i\,0.160X^{-0.618},$$

forms the rigid-backed surface impedance (Z_s=-iZ_c\cot(k_cd)) in the conventional (e^{+i\omega t}) convention, and conjugates it for the solver's (e^{-i\omega t}) convention. Its surface admittance (Y_s=1/Z_s) contributes

$$A_F \leftarrow A_F - i\rho\omega Y_s B_\Gamma,$$

where (B_\Gamma) is the consistent P1 surface mass matrix. This is a locally reacting approximation: it captures frequency-dependent absorption by a porous lining but not its lateral propagation, mounting gaps, compression, or frame elasticity. The UI defaults of 30 mm and 5,000 Pa·s/m² are a useful starting approximation for typical loose loudspeaker cabinet lining, not a material specification.

Only tetrahedra in the selected physical volume groups are retained. Each domain's vertices and facets are compacted, and boundary tags are remapped into a collision-free aggregate namespace before assembly.

A prescribed normal velocity (v_n) on a moving FEM surface is converted to the implemented pressure normal derivative

$$q_v = i\rho\omega v_n$$

and integrated against the triangular P1 boundary basis. Each excitation port uses (v_n=1\ \mathrm{m/s}) as its canonical basis input.

BEM region

The exterior uses the BEAT Engine Galerkin Burton-Miller formulation. Exterior boundary pressure (p_B) is represented in a continuous P1 space. The BEM Neumann data is represented facewise in a discontinuous DP0 space.

The interface introduces a P1 normal-derivative unknown (q_I), stored at the FEM interface vertices. Four sparse transfer operators connect the domains:

- (G_F), a consistent FEM boundary-mass operator that loads the FEM equation;
- (Q_B), which averages interface nodal derivatives onto BEM DP0 faces and applies the recorded normal-orientation signs;
- (T_F), which restricts FEM pressure to interface vertices;
- (T_B), which restricts BEM pressure to the corresponding interface vertices.

Electrodynamic transducer

Each electrodynamic component adds one rigid-piston velocity (v_d) and one voice-coil current (I) to the coupled unknown vector. The required direct parameters are:

- `re_ohm`;
- `le_h`;
- `bl_n_per_a`;
- `mmd_kg`, explicitly the dry moving mass;
- `cms_m_per_n`;
- `rms_n_s_per_m`;
- `motion_axis`, a three-component translation direction.

`Mms` is deliberately not accepted. Diaphragm area is integrated from the attached moving mesh surfaces, so a separate `Sd` is not required by the backend. The axis is normalized by the backend. For a triangle with acoustic domain outward normal (\mathbf n) and normalized motion axis (\mathbf d), the normal velocity and generalized acoustic force use the same projection:

$$v_n = (\mathbf{n}\cdot\mathbf{d})v_d, \qquad F_{\mathrm{ac}} = c_d\left[\sum_{\mathrm{FEM}}\int_{\Gamma_d} p(\mathbf{n}\cdot\mathbf{d})\,dS - \sum_{\mathrm{BEM}}\int_{\Gamma_d} p(\mathbf{n}\cdot\mathbf{d})\,dS\right].$$

This reciprocal projection represents a shaped but rigidly translating cone correctly. It also makes independently meshed front and rear FEM surfaces load the same degree of freedom with opposite pressure directions. Optional per-boundary signs are orientation corrections, not substitutes for the motion axis. The completion factor (c_d) is described under Symmetry.

With the solver's time convention, the electrical and mechanical impedances are

$$Z_e = R_e - i\omega L_e,$$

$$Z_m = R_{\mathrm{ms}} + i\left(\frac{1}{\omega C_{\mathrm{ms}}} - \omega M_{\mathrm{md}}\right).$$

The voltage and force equations are

$$Z_e I + Bl\, v_d = V,$$

$$Z_m v_d - Bl\, I - F_{\mathrm{ac}} = 0.$$

The reference voltage is currently fixed by the backend at (2.83\ \mathrm{V}) with zero source impedance. The same solved (v_d) generates every attached surface's projected normal derivative, so pressure loading, diaphragm motion, coil current, back-EMF, and radiation are solved bidirectionally.

Coupled block system

For the CPU production path and the FP64 reference path, each frequency uses the monolithic system

```
[ A_F     0      -G_F      [ p_F      [ f_v
    0   A_B   -R_BQ_B        p_B   =      0
  T_F  -T_B         0 ]     q_I ]        0 ].
```

Here (A_B) is the Burton-Miller pressure operator and (R_B) is its Neumann right-hand-side operator. The third block row enforces nodal pressure continuity. Flux continuity is built into the shared (q_I) unknown and its orientation-aware transfer to the BEM faces.

With (N_T) electrodynamic transducers, the monolithic system extends to

```
[    A_F      0     -G_F       -irhoomega D_F    0     [ p_F     [ f_v
       0    A_B  -R_BQ_B  -R_B(irhoomega D_B)    0       p_B         0
     T_F   -T_B         0                    0    0       q_I   =     0
  -B_F^T  B_B^T         0                  Z_m  -Bl       v_d         0
     0      0          0                   Bl  Z_e ]       I ]       V ].
```

(D_F) maps piston velocity into projected FEM surface loads and (D_B) maps it to BEM DP0 Neumann data. (B_F) and (B_B) contain the reciprocal projected pressure-force integrals,

including any reduced-driver completion factor. (A_F) is block diagonal when several bounded FEM domains are present. Opposite acoustic-domain force conventions make front and rear pressure act on the same mechanical degree of freedom.

If the FEM mesh has (N_F) vertices, the BEM mesh has (N_B) vertices, and the interface has (N_I) vertices, the monolithic matrix order is (N_F+N_B+N_I+2N_T).

The CUDA production path retains the union of FEM-BEM interface nodes and electrodynamic diaphragm nodes. All other FEM nodes are eliminated with an exact sparse Schur complement. If that retained set has (N_R) nodes, the dense coupled matrix has order (N_R+N_B+N_I+2N_T). For a prescribed-velocity-only model, (N_R=N_I), giving (N_B+2N_I). After the reduced system is solved, a backward sparse solve reconstructs every FEM domain's pressure. Static condensation changes the work and memory requirements, not the mathematical solution.

CPU production and FP64 reference solves remain monolithic. CUDA electrodynamic solves use the retained-surface condensed formulation.

The matrix is assembled and factored once per frequency. All requested excitation ports are then solved together as multiple right-hand sides, so an additional requested source adds a response column rather than another factorization.

Symmetry

Both production backends support:

- `off`: full model;
- `x`: positive-X half model, mirrored across (X=0);
- `xy`: positive-X/positive-Y quarter model, mirrored across (X=0) and (Y=0).

Electrodynamic components retain full-driver T/S parameters. Boundary Lab infers their symmetry representation from the perimeter topology of the selected moving surfaces on the reduced solve meshes. An active symmetry plane cuts a driver when the union of its selected surface groups has perimeter edges on that plane. Adjacent groups are unioned first, so internal seams between parts such as a dome and surround do not affect the result.

The inferred solver contract contains:

- `symmetry_role` is `complete_representative` when the fundamental domain contains one complete representative driver and `fractional_driver` when a symmetry plane cuts the same physical driver;
- `fractional_symmetry_axes` records which active planes cut that driver;
- `surface_completion_factor` is (2^n), where (n) is the number of those axes, and multiplies only the generalized pressure-force integral;
- `physical_driver_orbit_count` records the number of distinct identical physical drivers represented by symmetry images.

The component editor displays this inference instead of asking the user to choose a

representation. The compiler repeats the inference, replacing any legacy persisted values so a change in global symmetry cannot leave stale component scaling. Disconnected selected patches that imply different cuts are rejected as ambiguous.

The completion factor times the orbit count equals 1, 2, or 4 for off, X, or XY symmetry respectively. A motion axis must lie in every symmetry plane that cuts the same physical driver. Velocity and Neumann data are not multiplied: BEM symmetry images already reconstruct their acoustic effect. Velocity and current outputs are per physical driver; aggregate current for an orbit is the reported current times `physical_driver_orbit_count`.

When the component editor infers a motion axis automatically, it first reconstructs the selected diaphragm's normal-orientation tensor across every inferred fractional symmetry axis. This removes the lateral bias that can otherwise arise from averaging only a reduced half or quarter of a curved diaphragm and guarantees that the inferred rigid-translation axis lies in each plane that cuts the physical driver.

Both FEM and BEM meshes must lie in the selected positive fundamental domain. The FEM cut faces have the natural zero-normal-derivative condition represented by a rigid symmetry surface. BEM operator assembly includes the reflected source contributions, and exterior-field evaluation includes all symmetry images. Observation points can therefore cover the full field even though the solved mesh is reduced.

Interface construction is symmetry-aware: a reduced interface may meet a symmetry plane, and a reduced BEM surface may remain open on that plane. Open edges away from an active symmetry plane are rejected.

Symmetry participation is determined separately for each interface. An interface whose perimeter contains edges on an active symmetry plane is treated as an open reduced patch on those planes; a complete port mouth away from the planes retains its ordinary closed perimeter even when the project uses X or XY symmetry. This allows one reduced model to combine central, symmetry-cut openings with complete side or top openings.

Symmetry reduces the FEM, BEM, and interface unknown counts before the coupled matrix is built. This is especially valuable because the BEM operators and the final coupled factorization are dense.

Excitations, channels, and application results

For each frequency, the backend computes one complex response for every requested excitation-port ID. The application currently requests exterior pressure at:

- the horizontal polar observation points;
- the vertical polar observation points;
- optional Fibonacci-sphere points used by balloon plots.

Responses assigned to the same application channel are summed before ordinary channel synthesis. Gain, polarity, delay, high-pass and low-pass filters, and plot normalization are applied after the physical solve. Changing those settings does not change the coupled matrices.

The backend protocol also recognizes these quantities for validation and specialized callers:

Quantity	Unit	Array axes
`fem_nodal_pressure`	Pa	excitation, FEM node
`bem_boundary_pressure`	Pa	excitation, BEM node
`interface_normal_derivative`	Pa/m	excitation, interface node
`diaphragm_velocity`	m/s	excitation, transducer
`voice_coil_current`	A	excitation, transducer
`exterior_pressure`	Pa	excitation, observation

The main application path currently requests only `exterior_pressure` and does not yet project transducer velocity, current, or derived electrical impedance into its plots. Impedance fields in the legacy live-result shape remain unavailable.

Diagnostics

Every production frequency result identifies its precision, BEM backend, linear backend, symmetry mode, formulation, linear solver, full system order, solved system order, and detailed stage timings. It also reports the maximum across excitation bases of:

- relative pressure-continuity error at the interface;
- relative integrated interface-flux conservation error.

For multi-port models these are the worst values over the individual interfaces, not one potentially cancelling aggregate. Per-interface IDs and error arrays are included in the diagnostics.

Production solves deliberately do not retain the complete coupled matrix after factorization. Consequently, they do not report a full coupled-system residual. The FP64 reference path keeps the matrices and additionally reports:

- the monolithic relative residual;
- error against a separate BEM-only replay driven by the solved interface Neumann data.

This distinction matters when reading status messages or benchmark output: the interactive application normally shows interface continuity, while validation runs can show a full residual.

Performance and practical limits

The application keeps a Julia worker alive between solves. The first solve in a session includes Julia loading and compilation; later solves reuse the worker. Within one frequency sweep, Boundary Lab caches frequency-invariant data, including:

- FEM stiffness and mass matrices;

- interface transfer operators;
- BEM P1 and DP0 spaces;
- quadrature and singular-correction geometry;
- symmetry-image and field-evaluation geometry;
- CPU or GPU assembly support data.

The frequency-dependent FEM Helmholtz matrix, BEM operators, coupled blocks, and factorizations are rebuilt at every frequency.

The solver is still limited by dense BEM and coupled algebra. BEM assembly grows approximately quadratically with boundary size. CPU monolithic LU grows approximately cubically with (N_F+N_B+N_I+2N_T). CUDA condensation removes FEM interior nodes from the dense block, but the remaining interface, diaphragm, BEM, and transducer system is still dense and requires substantial GPU memory. Symmetry can reduce these costs dramatically for both prescribed-velocity and electrodynamic models. The current backend is not an iterative or large-scale fast-multipole solver.

Before committing to a fine sweep, test a few representative frequencies and watch the reported system order, assembly time, factorization time, and available host or GPU memory.

Unsupported physics

The physical-system schema includes roles intended for future solvers. The following are not implemented by the current coupled backend and are normally rejected during application preparation or backend validation:

- general impedance boundary kinds or arbitrary impedance functions (the Miki rigid-backed treatment on bounded rigid walls is supported);
- spatially varying acoustic material loss within one FEM region (one homogeneous bulk-loss factor per bounded region is supported);
- `Mms` input or automatic conversion from conventional T/S parameter sets;
- passive radiators;
- nonideal amplifier/source impedance;
- cone breakup, nonlinear or asymmetric `Bl`, thermal effects, and lossy or frequency-dependent voice-coil inductance;
- nonuniform prescribed-motion profiles;
- more than one unbounded exterior region;
- different fluid properties among coupled acoustic regions;
- iterative, fast-multipole, or distributed coupled solution methods;
- Bempp, ROCm, server, or exterior-local execution for a coupled physical system.

Validation and profiling

The Julia smoke suite checks FEM matrices, a prescribed-velocity interior solve, a sealed-cavity mode, interface operators, and—when enabled—the full coupled fixture:

```
$env:BLAB_RUN_COUPLED_REFERENCE = "1"
julia --project=src/blab/solvers/julia_local `
  src/blab/solvers/julia_local/scripts/smoke_coupled_solver.jl
```

The dedicated noncubic-cavity correctness test compares the sparse P1 FEM matrices with the analytic rigid-wall modes of a 470 x 330 x 220 mm cavity. Centered patches on the X, Y, and Z walls verify first-axial-mode source selectivity at nominal 20, 14, and 10 mm element sizes. A sparse block shift-invert solve checks that the first twelve nonzero modes match the analytic rectangular-cavity sequence and converge monotonically under refinement. The test also checks a homogeneous passive bulk-loss factor in the interior helper, including the solver sign convention, resonant amplitude scaling, half-power bandwidth, and modal Q for all three axes:

```
julia --project=src/blab/solvers/julia_local `
  src/blab/solvers/julia_local/scripts/test_noncubic_cavity_loss.jl
```

The companion high-frequency dispersion diagnostic samples exact axial and oblique modes from about 2 to 10 kHz on the same three mesh densities:

```
julia --project=src/blab/solvers/julia_local `
  src/blab/solvers/julia_local/scripts/analyze_noncubic_ppw.jl
```

For these P1 tetrahedral fixtures, conservative sampled limits are about 17 nominal elements per wavelength for 1% modal-frequency error, 12 for 2%, 8 for 5%, and 6 for 10%. The nominal 14 mm mesh is therefore near the 5% boundary at 3 kHz and is not a reliable 5--10 kHz reference. Bulk loss can reduce resonance Q, but it cannot correct this spatial-dispersion error.

The curved-interface production fixture has a separate targeted convergence diagnostic. It compares the baseline and independently exported detailed FEM meshes using overlapping block shift-invert slices, nearest-node transferred modal assurance, and HF, MF, interface, and wall participation:

```
julia --project=src/blab/solvers/julia_local `
  src/blab/solvers/julia_local/scripts/analyze_curved_fem_convergence.jl
```

Pass `--stats-only` to validate geometry, physical groups, edge lengths, and tetrahedron quality without running the eigensolves. For the current fixtures, the detailed mesh reduces median edge length from 3.18 to 1.96 mm while changing enclosed volume by 0.060%. Targeted modes from 3 to approximately 10.2 kHz pair across the meshes. Median baseline-to-detailed frequency shifts are 0.20% from 3--5 kHz, 0.41% from 5--8 kHz, and 0.70% above 8 kHz; the corresponding maximum sampled shifts are 0.44%, 0.59%, and 0.86%. Closely spaced modes can rotate within their shared modal subspace, so low individual MAC or participation agreement in a cluster must be assessed at the subspace level before declaring a mode unmatched.

For the solver's `exp(-i omega t)` convention, this diagnostic loss uses

$$A_F(eta) = K - k^2(1 + ieta)M = K - k^2M - ietak^2M.$$

The end-to-end benchmark exercises the compiled request and streamed-result path. This CPU example uses the same FP32 monolithic formulation as an interactive CPU application solve:

```
python scripts/benchmark_coupled_solver.py `
  --julia C:\path\to\julia.exe `
  --mode interactive `
  --precision float32 `
  --bem-backend cpu `
  --persistent `
  --repeat 2
```

For CPU precision studies, the dedicated comparison runs identical meshes, quadrature, excitations, and field points through `Float64/ComplexF64` and `Float32/ComplexF32`:

```
julia -t 4 --project=src/blab/solvers/julia_local `
  src/blab/solvers/julia_local/scripts/compare_coupled_precision.jl
```

The comparison reports relative complex-vector errors plus magnitude and phase deltas for FEM pressure, BEM pressure, interface derivative, and exterior pressure. Values far below each quantity's peak are excluded from magnitude and phase summaries so response nulls do not dominate the statistics.

CUDA Server Docker Image

Boundary Lab's BEAT Engine CUDA solve server can be packaged as a GPU container. Future support for AMD ROCm Docker images is planned.

Build

From the repository root:

```
docker build -f docker/server-cuda.Dockerfile -t boundary-lab-server:cuda
```

The build installs Python dependencies, Julia, the `src/blab/solvers/julia_cuda` project, and a CUDA-focused Julia sysimage at `/app/blab-beat-cuda.so`. To skip the sysimage while keeping Julia precompilation, add `--build-arg BLAB_BUILD_SYSIMAGE=0`.

Run Locally On A GPU Host

```
docker run --rm --gpus all \
  -p 8765:8765 \
  -v blab-server-data:/data \
  boundary-lab-server:cuda
```

Check the server:

```
curl http://127.0.0.1:8765/health
```

The response should report `"solver":"beat_cuda"`.

The container listens on port `8765` by default. Authentication is optional; without

`BLAB_AUTH_TOKEN`, only expose it on localhost or a trusted private network.

Runpod HTTPS Deployment

For an internet-reachable Runpod Pod:

1. In Boundary Lab Preferences, set `BEM Solver` to `Server`.
2. Click `Generate` beside `Server access token`, then click `Copy`.
3. Create a Runpod Secret such as `boundary_lab_access_token` using the copied token as its value.
4. In the Pod template, set:
   `BLAB_AUTH_TOKEN={{ RUNPOD_SECRET_boundary_lab_access_token }}`
5. Expose `8765` as an HTTP port, not a raw TCP port.
6. Set Boundary Lab's server URL to:
   `https://<pod-id>-8765.proxy.runpod.net`
7. Click `Check Server`, then accept Preferences.

Runpod terminates HTTPS at its HTTP proxy. The bearer token authenticates requests at the Boundary Lab server. Do not put the token directly in the Docker image, Pod template text, server URL, or command line.

Configuration

The entrypoint reads these environment variables:

Variable	Default
BLAB_SERVER_HOST	0.0.0.0
BLAB_SERVER_PORT	8765
BLAB_SERVER_SOLVER	beat_cuda
BLAB_JULIA_EXECUTABLE	/opt/juliaup/bin/julia
BLAB_JULIA_THREADS	auto
BLAB_JULIA_SYSIMAGE	/app/blab-beat-cuda.so
BLAB_JULIA_CPU_TARGET	generic,+aes
BLAB_WARM_SOLVER	off
BLAB_MAX_RUNNING_JOBS	1
BLAB_MAX_QUEUED_JOBS	4
BLAB_MAX_REQUEST_MB	20

BLAB_MAX_ASSET_MB	14
BLAB_MAX_FREQUENCIES	1000
BLAB_JOB_RETENTION_HOURS	24
BLAB_EVENT_STREAM_WINDOW_SECONDS	25
BLAB_AUTH_TOKEN	empty; authentication disabled
BLAB_LOG_LEVEL	INFO
BLAB_ARTIFACT_DIR	/data/server_jobs

Example override:

```
docker run --rm --gpus all \
  -e BLAB_JULIA_THREADS=8 \
  -e BLAB_WARM_SOLVER=tiny \
  -e BLAB_ARTIFACT_DIR=/data/jobs \
  -p 8765:8765 \
  -v blab-server-data:/data \
  boundary-lab-server:cuda
```

`BLAB_WARM_SOLVER=worker` starts the persistent Julia worker during server startup. `BLAB_WARM_SOLVER=tiny` also runs a one-frequency tetrahedron solve, which is slower to start but warms more CUDA/JIT paths before the first client job. The sysimage is built with `BLAB_JULIA_CPU_TARGET=generic,+aes`, which avoids host-specific targets while keeping AES-NI available for dependencies that emit AES intrinsics. Set `BLAB_JULIA_SYSIMAGE=` to disable the bundled sysimage for diagnostics.

The 20 MiB request limit accommodates typical Boundary Lab meshes while bounding memory use before JSON parsing. Base64 expands uploaded files, so the decoded asset limit defaults to 14 MiB. Event responses rotate every 25 seconds and the GUI resumes from the last event, avoiding long-lived proxy connection limits. Terminal jobs and artifacts expire after 24 hours unless retention is set to zero.

To run a shell instead of the server:

```
docker run --rm -it --gpus all boundary-lab-server:cuda bash
```

Physical System Model

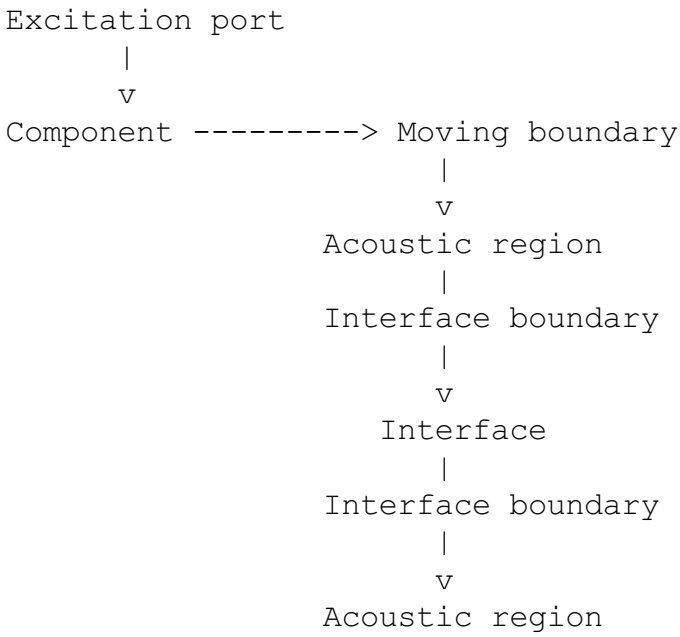
Boundary Lab represents a loudspeaker as one physical system rather than as a collection of independent BEM, FEM, and lumped-element simulations. The model describes the air domains, the surfaces around those domains, the devices that move some of those surfaces, and the connections that allow sound to pass between domains.

The central concepts are:

- **Regions** contain acoustic media.

- **Boundaries** assign physical behavior to region surfaces.
- **Interfaces** connect boundary surfaces belonging to different regions.
- **Components** represent devices that drive or respond to moving boundaries.
- **Excitation ports** identify the independent physical inputs used to compute unit-response transfer functions.

These concepts form a graph:



The editable graph is stored in a Boundary Lab project. Before solving, the physical-system compiler resolves its named mesh groups, validates its relationships, and produces an immutable solver contract.

Application workflow

The main application exposes the physical model through a tabbed **System** window:

- **Regions** assigns a name, bounded-interior or unbounded-exterior type, mesh, and (for bounded regions) physical volume group. It also owns the exterior-region mesh-stitching choice.
- **Boundaries** assigns each tagged surface as rigid, moving, or interface. New surfaces default to rigid, and legacy unused assignments migrate to rigid when loaded into the editor.
- **Interfaces** uses **Build/Identify Interfaces** to match bounded and unbounded interface sides. When an imported BEM interface is meshed differently, Boundary Lab writes a derived BEM mesh whose interface nodes and faces come from the FEM side, then validates conformity and orientation. Curved interface interiors are supported when the FEM and BEM perimeters describe the same planar opening; the surrounding planar BEM surface may be split across multiple Gmsh geometrical entities.

- **Components** attaches prescribed-velocity or electrodynamic components to moving boundaries, sets per-surface relative velocity, and assigns an application channel.

The **Meshes** window remains responsible for mesh import or replacement, scale, translation, and global symmetry. Tetrahedral imports bypass the surface-only BEM cleaner so their volume connectivity and physical groups are preserved. Replacing an imported row preserves its mesh identity, which keeps downstream references intact when the replacement uses the same physical-group names.

The application infers the solve kind from the authored regions. Exactly one unbounded region with no bounded regions is an exterior BEM system; adding one or more bounded regions selects the coupled FEM-BEM path and enables the Interfaces tab. Exterior-only systems currently accept prescribed-velocity components, while coupled systems may also use electrodynamic transducers.

Component-to-channel routing is stored as application project state, not in the physical system or compiled solver contract. The coupled solver returns one complex reference response per excitation port; responses routed to the same channel are combined before the existing channel gain, polarity, delay, and filter settings are applied.

Regions

A region represents a connected acoustic domain with material properties such as density, sound speed, and an optional loss model. A bounded FEM region may define a dimensionless `bulk_loss_factor`. The factor is constant within that region and may differ between disconnected interior chambers.

The initial model supports two region kinds:

- `bounded_air`: a finite air volume solved with FEM. Its mesh contains tetrahedra and one or more tagged physical volume groups.
- `unbounded_air`: the exterior acoustic domain solved with BEM. Its mesh contains the triangular surface bounding the exterior problem.

For example, a vented enclosure may contain:

```
region:enclosure     bounded_air      FEM tetrahedral mesh
region:exterior      unbounded_air    BEM triangular mesh
```

A region owns the interpretation of its boundaries. The same physical device may consequently have one boundary facing the enclosure region and another facing the exterior region.

Regions do not describe how individual surfaces behave. That is the role of boundary assignments.

Boundaries

A boundary assigns a physical role to one tagged surface group on one mesh. It always belongs to exactly one region.

The model schema defines these boundary roles:

- `rigid`: zero normal surface velocity.
- `moving`: motion is supplied by a component.
- `interface`: acoustic state is coupled to another region through an interface.
- `impedance`: pressure and normal velocity obey a surface-impedance model.
- `unused`: a compatibility/future-facing schema value that is not offered by the current System editor. Existing unused assignments are presented and saved as `rigid`.

These are model-level roles. The current UI creates `rigid`, `moving`, and `interface` assignments; it represents the supported Miki wall treatment as parameters on a rigid boundary rather than as a generic `impedance` role. A solver backend must declare which roles it supports, so a future-facing schema value does not imply that the current numerical backend can solve it.

The coupled backend additionally supports a locally reacting porous lining on a bounded region's `rigid` boundary. It remains a rigid boundary assignment and stores a `wall_impedance` parameter object containing the `miki` model, `thickness_m`, and `flow_resistivity_pa_s_per_m2`. Keeping the assignment rigid means disabling the optional treatment returns the same surface to the hard-wall condition without changing its physical role.

An example boundary assignment is:

```
{
  "id": "boundary:rear-diaphragm",
  "name": "Driver rear diaphragm",
  "region_id": "region:enclosure",
  "group": {
    "mesh_id": "mesh:enclosure",
    "dimension": 2,
    "name": "Radiator"
  },
  "kind": "moving",
  "parameters": {}
}
```

This says that the `Radiator` surface group is part of the enclosure region and that its normal motion comes from a physical component.

Boundaries are deliberately region-local. A surface on an FEM mesh and its matching surface on a BEM mesh are separate boundary assignments even when they occupy the same position.

The compiler resolves each group name to its Gmsh physical tag and element count. Every tagged physical surface used by a region must receive exactly one boundary role. This prevents an unassigned surface from silently becoming an unintended opening or rigid wall.

Interfaces

An interface connects two boundary assignments and transfers acoustic state between their regions. It represents a relationship between surfaces rather than another physical surface.

Each FEM-BEM interface has:

- one boundary belonging to a bounded FEM air region;
- one boundary belonging to the unbounded BEM exterior;
- both boundaries assigned the `interface` role;
- a coordinate tolerance used during conformity validation;
- compiled vertex, face, and normal-orientation mappings.

For example:

```
{
  "id": "interface:port",
  "name": "Enclosure port",
  "bounded_boundary_id": "boundary:fem-port",
  "unbounded_boundary_id": "boundary:bem-port",
  "coordinate_tolerance_m": 1e-8
}
```

The two referenced boundaries may use different mesh tags and different global vertex numbers. The compiled interface records their explicit correspondence.

At the numerical level, the interface enforces pressure continuity:

$$p_{\mathrm{FEM}} = p_{\mathrm{BEM}}$$

and conservation of normal velocity or acoustic flux. When both normals point outward from their respective regions, this is commonly written as:

$$v_{n,\,\mathrm{FEM}} + v_{n,\,\mathrm{BEM}} = 0$$

Coordinate matching alone is not sufficient. The coupled solver must also transfer quantities correctly between the FEM and BEM basis spaces.

An ordinary open port mouth is therefore an interface, not a component. It connects two air domains but does not introduce an independent mechanical device.

Several bounded FEM chambers may connect to different tagged openings in the same unbounded BEM mesh. When coplanar openings require rebuilding, Boundary Lab preserves the other interface groups while conforming each FEM-BEM pair.

Components

A component represents a physical device with behavior beyond the air itself. It acts through one or more `moving` boundaries.

The initial component kinds are:

- `ideal_velocity_source`: prescribes motion directly for acoustic validation.
- `electrodynamic_transducer`: couples electrical, mechanical, and acoustic behavior.
- `passive_radiator`: has mechanical dynamics and acoustic loading but no electrical drive.

The coupled backend supports ideal velocity sources and a first linear electrodynamic model. Passive-radiator behavior remains deferred. The System editor exposes both supported active component types.

An ideal source might be authored as:

```
{
  "id": "component:driver",
  "name": "Ideal driver",
  "kind": "ideal_velocity_source",
  "boundary_ids": ["boundary:rear-diaphragm"],
  "parameters": {
    "motion_profile": "uniform"
  }
}
```

The component owns the behavior of its referenced moving boundaries. A moving boundary must be owned by exactly one component so that two devices cannot silently prescribe incompatible motion on the same surface.

Each moving boundary may also have a positive relative motion weight. The component editor displays this value in dB while the physical model stores its linear amplitude in `boundary_motion_weights`. A dome, inner surround, and outer surround can therefore share one component while using progressively lower surface velocity. Electrodynamic coupling applies the same weight to boundary velocity and generalized pressure force so the coupling remains reciprocal.

A component may reference multiple boundaries, including moving groups from different FEM meshes. A full electrodynamic driver, for example, may have independently triangulated front- and rear-chamber diaphragm boundaries:

```
Front FEM chamber --- Front diaphragm boundary
                                    |
Excitation port -------- Driver component
                                    |
Rear FEM chamber ---- Rear diaphragm boundary
```

Both acoustic pressures load the same mechanical diaphragm degree of freedom. The diaphragm surfaces are not acoustic interfaces and do not require matching nodes: their pressures remain independent and couple only through the component. The component converts the resulting pressure difference, motor force, moving mass, suspension, and damping into boundary motion.

The initial electrodynamic backend model is a single-axis rigid translation with direct `re_ohm`,

`le_h`, `bl_n_per_a`, `mmd_kg`, `cms_m_per_n`, and `rms_n_s_per_m` parameters plus a three-component `motion_axis`. `mmd_kg` is dry moving mass; `Mms` is not accepted. Normal velocity and generalized force both use each face's projection onto the motion axis. Diaphragm projected area and pressure force are therefore integrated from the attached moving meshes, including shaped rigid cones.

Reduced electrodynamic models keep full physical-driver T/S parameters. The compiler determines whether a driver is cut by each active symmetry plane by examining perimeter edges in the union of its selected moving surface groups. It then emits these derived component parameters into the solver contract:

- `symmetry_role`;
- `fractional_symmetry_axes`;
- `surface_completion_factor`;
- `physical_driver_orbit_count`.

The completion factor scales only the recovered full-diaphragm pressure force. The orbit count describes distinct identically driven physical drivers and is used when aggregating electrical current or power. These fields may exist in older project files, but current compilation re-infers and replaces them from the active symmetry mode and reduced mesh topology.

A passive radiator uses the same general pattern without an excitation port. Its motion is driven by the pressure difference across its front and rear boundaries.

Excitation Ports and Application Synthesis

An excitation port identifies an independently driven physical component input. It does not contain gain, polarity, delay, crossover filters, or channel routing.

For an ideal velocity source, the port represents a canonical normal-velocity input:

```
{
  "id": "excitation:woofer",
  "name": "Woofer unit normal velocity",
  "component_id": "component:woofer",
  "kind": "normal_velocity"
}
```

For an electrodynamic transducer, the port represents a canonical voltage input. The current reference amplitudes are `1 m/s` for `normal_velocity` and `2.83 V` for `voltage`. Passive radiators have no excitation port.

The coupled solver computes one complex reference response for each requested port. For a linear system, the pressure at an observation point is then:

$$p(\mathbf{x}, f) = \sum_j H_j(\mathbf{x}, f) u_j(f)$$

where `H_j` is the solved response for excitation port `j` and `u_j` is an application-side complex weight. Gain, polarity, delay, ideal crossover filters, equalization, and channel mixing are all included in `u_j` after the physical solve.

The separation is:

```
Physical system                 Application synthesis
---------------                 ---------------------
excitation:woofer  <----------  channel assignment
      |                         gain / polarity / delay
component:woofer               HPF / LPF / EQ
      |
boundary:woofer-diaphragm
```

Application synthesis presets remain editable project state, but they are not part of the compiled physical system or numerical solve request. Changing an ordinary DSP setting therefore does not invalidate a completed basis solve.

Passive electrical crossover networks, amplifier source-impedance interaction, feedback control, and nonlinear or level-dependent processing would alter the physical system rather than merely its excitation weights. They are outside the intended Boundary Lab coupled-model scope.

Complete Vented-Enclosure Example

Consider an enclosure represented by a tetrahedral FEM mesh and an exterior surface represented by a BEM mesh:

```
excitation:woofer
      |
      v
component:woofer
      |
      +---- boundary:front-diaphragm (moving) ---- region:exterior
      |
      +---- boundary:rear-diaphragm  (moving)
                        |
                        v
                region:enclosure
                        |
                boundary:fem-port
                        |
                  interface:port
                        |
                boundary:bem-port
                        |
                        v
                region:exterior
```

The remaining enclosure surfaces receive rigid boundary assignments. The exterior cabinet surfaces also receive rigid assignments.

During a coupled solve:

1. The solver applies the excitation port's canonical reference input.
2. The component determines or prescribes diaphragm motion for that basis.
3. The front diaphragm excites the exterior while the rear diaphragm excites the enclosure FEM region.
4. Enclosure pressure and velocity reach the FEM side of the port.
5. The port interface transfers pressure and flux to the BEM side.
6. The BEM region computes the combined exterior loading and radiation.
7. In a dynamic transducer model, the front and rear acoustic loads feed back into the component and the complete system is solved simultaneously.
8. Boundary Lab applies channel routing and DSP to the stored complex basis responses in the application.

Common Classification Questions

Is a diaphragm a component?

The diaphragm surface is a boundary. The driver or passive radiator that determines its motion is the component.

Is a port a component?

An explicitly meshed air opening between FEM and BEM regions is an interface. A reduced-order port model with inertance, resistance, or nonlinear behavior could later be represented as a component or acoustic multiport.

Is an enclosure wall a component?

An acoustically rigid wall is a boundary. A flexible enclosure panel would require a structural component or structural region that is outside the initial pressure-acoustics model.

Does an interface own a mesh?

No. Its two boundary assignments refer to mesh surface groups. The interface owns only their connection and compiled topology mapping.

Can a component connect two regions?

Yes, indirectly. It references a moving boundary in each region and supplies the shared device equations that relate their motion and acoustic loading.

Authoring Model and Compiled Model

The saved project contains human-oriented references:

```
mesh group name: "Interface"
boundary role:   "interface"
```

```
connected to:    "boundary:bem-port"
```

The compiler converts these into solver-oriented data:

```
physical tag:       3
element count:      180
vertex map:         FEM vertex -> BEM vertex
face map:           FEM facet -> BEM facet
orientation signs: +1 or -1
```

The solver receives the compiled model together with frequencies, requested unit-basis excitations, output quantities, and numerical options. Each basis-dependent result declares its excitation IDs and an `excitation` array axis. The compiled model contains no application DSP settings.

Keeping authoring, compiled, and synthesis models separate allows project files, numerical backends, and future physics features to evolve independently.

Model Invariants

The compiler should reject a system when:

- an object identifier is missing or duplicated;
- a region references a mesh of the wrong purpose;
- a bounded region has no tetrahedral physical volume;
- a tagged surface has no boundary assignment or multiple assignments;
- a moving boundary is not owned by exactly one component;
- a component references a non-moving boundary;
- an active component does not have exactly one compatible excitation port;
- a passive component has an excitation port;
- an interface references a boundary that is not marked as an interface;
- the two sides of a FEM-BEM interface are not conforming;
- mesh scale, placement, material properties, or excitation-port kinds are invalid.

These checks keep geometry mistakes and product-model mistakes out of the numerical backend, where they would otherwise be harder to diagnose.